ASPIRE TECH · CRM MIGRATION PROGRAMME

Master Runbook

GoHighLevel → Twenty + n8n.
Audit, build, deploy, migrate, operate, hand over.
Every command, every gate, every failure mode.

Status Current as of 25 August 2026

Owner Rafiur Rahman (project lead / primary operator)

Second operator Not yet named — this is an open risk. See §17.

Source docs/RUNBOOK.md — this PDF is generated from it

Repository github.com/rafilovestosuffer/aspire-crm-twenty (still public — see §16.3)

Classification Internal. Contains hostnames, procedures and operational detail. No secrets, and none may ever be added.

> This document is generated. Edit **docs/RUNBOOK.md** and re-run **docs/build_runbook_pdf.py** — never the PDF. An out-of-date runbook is worse than none, because it is trusted.

Contents

Sections start on a new page. Reading paths by role are in §0.2; every gate is indexed in §20 B.

§0 · How to use this runbook 5

- 0.1 What this document is 5
- 0.2 Reading paths 5
- 0.3 Conventions used here 5
- 0.4 The document map 5

§1 · The project in plain language 7

- 1.1 One paragraph 7
- 1.2 Why we are doing it — three reasons, in order 7
- 1.3 What we are not claiming 7
- 1.4 The deliverables 7

§2 · Scope, decisions and open questions 9

- 2.1 In scope 9
- 2.2 Out of scope, deliberately 9
- 2.3 Decisions on record 9
- 2.4 Open questions still outstanding 10

§3 · Architecture 11

- 3.1 The shape of it 11
- 3.2 Division of labour 11
- 3.3 The capability boundary — the hard ceiling 12
- 3.4 The nevers 12

§4 · Part A — The GoHighLevel audit 14

- 4.1 Governing principles 14
- 4.2 The six tiers 14
- 4.3 Tier 0 — access, token, scopes 14
- 4.4 Verify auth by hand before running anything 15
- 4.5 Tier 1 — the structural pull 15
- 4.6 What the 18 August 2026 pull found 16
- 4.7 Tier 4 — workflow internals, and why they are not available 18
- 4.8 Tier 5 — the manual sweep 19
- 4.9 Tier 6 — build the report 19
- 4.10 What the audit cannot tell you 20
- 4.11 Closing the audit down 20

§5 · Prerequisites and environment 21

- 5.1 Laptop (demo / development) 21
- 5.2 Server 21
- 5.3 Things only Aspire can supply 21
- 5.4 The environment file 22

§6 · The object model 24

- 6.1 The idea in one line 24
- 6.2 The objects that carry the business 24
- 6.3 The full mapping — GHL feature → Twenty object → who executes 25
- 6.4 Provisioning it 25
- 6.5 What a person still has to decide 26

§7 · The automation library 27

- 7.1 The seventeen 27
- 7.2 The two dev-only workflows 28
- 7.3 Design rules every workflow follows 28
- 7.4 Deploying them 28

§8 · Part B — Building the stack on a laptop 30

- 8.1 The whole thing in one command 30
- 8.2 Step 1 — start the stack 30

8.3 Step 2 — verify every layer 31
8.4 Steps 3–4 — workspace and API keys 31
8.5 Step 6 — seed the demo data 31
8.6 Step 5 revisited, and native Twenty configuration 32
8.7 Steps 8–10 — the three gates, and why each exists 32
8.8 What a passing suite still does not prove 33
8.9 Where things are, on a laptop 34
8.10 The live Gmail demo (optional, laptop only) 34
8.11 The demo — ten minutes 35
§9 · Part C — Deploying to the company server 37
9.1 Before you start, have these in your hand 37
9.2 The sequence 37
9.3 What init-vps-env.sh does, and the four things it cannot know 38
9.4 The editor allowlist — the largest avoidable risk here 38
9.5 Rehearse the certificate before the real one 38
9.6 preflight.sh — twenty seconds that save an evening 39
9.7 deploy.sh — the eleven gated steps 39
9.8 What deploy.sh does not do 40
9.9 The five things to do immediately after 40
9.10 Post-deploy verification 40
§10 · Deploying to an internal (office-network) server 42
10.1 What an internal server can and cannot do 42
10.2 The differences from §9 42
10.3 The internal-CA trap 42
§11 · Part D — Data migration 43
11.1 Order of operations 43
11.2 The consent export — the highest-severity item in this project 43
11.3 Stage mapping 43
11.4 Field mapping 43
11.5 Reconciliation 44
11.6 What does not migrate 44
§12 · Part E — Parallel run and cutover 45
12.1 Parallel run 45
12.2 Pre-cutover checklist 45
12.3 Cutover 45
12.4 Rollback 45
§13 · Part F — Operating it 47
13.1 The operating calendar 47
13.2 Backups and restore 47
13.3 The watchdog 48
13.4 Monitoring — what to watch and why 48
13.5 Routine operator tasks 48
13.6 Diagnostics — the first five commands 49
§14 · Troubleshooting 50
14.1 Build and stack 50
14.2 The metadata 500 — read this before blaming the API key 50
14.3 Workflows and queries 51
14.4 Mail 52
14.5 GoHighLevel 52
14.6 Certificates and proxy 52
§15 · The quirks that cost us time 54
15.1 Twenty REST query syntax 54
15.2 A page is 200 rows and nothing raises that 54
15.3 Dates go to Twenty as UTC with a Z, always 54
15.4 Every interpolated filter value must be encodeURIComponent-wrapped 55

15.5 Metadata is a slice, not a schema 55
15.6 n8n 55
15.7 Twenty data-model facts 55
15.8 GoHighLevel 55
§16 · Security, privacy and compliance 56
16.1 The governance position 56
16.2 Secrets 56
16.3 Network exposure 56
16.4 Two environments, not one 57
16.5 Consent as a control, not a field 57
§17 · Handover and the second operator 58
17.1 The risk, stated plainly 58
17.2 The exit criterion 58
17.3 What the second operator must be able to answer 58
17.4 Hard questions, and the honest answers 58
§18 · Status — measured, not asserted 59
18.1 What is built 59
18.2 Feature disposition — all 111 catalogued 59
18.3 What the 33 deferred actually are 59
18.4 What has been rehearsed rather than assumed 59
18.5 What has not been verified against a live stack 60
§19 · Cost, roadmap and the things that can hurt 61
19.1 Cost 61
19.2 Roadmap 61
19.3 The three things that can actually hurt 62
19.4 Sequencing risks, if the deferred items ever come back 62
§20 · Appendices 63
A · Command index 63
B · Gate index — every point where you stop 64
C · Glossary 65
D · Change control for this document 65

§0 · How to use this runbook

0.1 What this document is

This is the whole project in one file: what we are doing, why, what has been built, every command that builds it, every command that proves it, how it goes to the server, how the data moves, how it is operated afterwards, and what to do when each part fails.

It is written so that **a competent operator who has never seen this project can run it from this file alone**. That is not a stylistic goal. It is the Phase 5 exit gate: one person who can restore this system is a single point of failure, and it is the most likely thing to hurt us.

0.2 Reading paths

Do not read it end to end unless you are the second operator being brought up. Take the path that matches your job today:

You are	Read
Explaining the project to management	§1, §2, §18, §19
Building the demo stack on a laptop	§5, §6, §7, §8
Deploying to the company server	§5.2, §9, §10
Migrating real data	§11
Cutting over from GoHighLevel	§12
Operating it week to week	§13, §14
On call, something is broken	§14 (find the symptom), then §13.6
Auditing GoHighLevel again	§4
Being handed the system	Everything, in order, with a keyboard

0.3 Conventions used here

- `$` prefixed commands are typed by a person. Output shown is what you should see; anything else is a failure, not a variation.
- **Gate** means: do not proceed until this passes. Gates exist because the failure they catch is silent or expensive later. There are no optional gates.
- **Never** means never — these are enumerated in §3.4 and each one has a specific incident behind it.
- Paths are relative to the repository root unless stated.
- Where a fact was verified against a live system, the date and target are given. Where it was not, it says so. **Unverified is not the same as false, and neither is the same as true.**

0.4 The document map

This runbook is the spine. The detail documents are the ribs, and they stay authoritative for their own subject:

Document	Authoritative for
`CLAUDE.md`	Operating rules, and the hard-won API quirks (reproduced as §15)
`docs/00-audit-method.md`	Audit method, the six tiers, disposition rules
`docs/01-scope-and-questions.md`	Open decisions Q1–Q8
`docs/02-ghl-api-findings.md`	What the GHL API can and cannot do, with sources
`docs/03-replacement-stack.md`	The eight vendor clusters and their lead times
`docs/04-manual-sweep.md`	Tier 5 checklist — everything no API serves
`docs/05-runbook.md`	The GHL audit pull, operator-level (expanded here as §4)
`docs/06-twenty-object-model.md`	The object model and its two arguable objects
`docs/07-build-plan.md`	Sprint plan and production requirements
`docs/08-local-build.md`	Laptop build detail and the Gmail demo
`docs/09-status-and-handover.md`	Measured status, cost, hard questions
`docs/10-vps-deployment.md`	Public server deploy (Contabo)
`docs/11-internal-server-deployment.md`	Office-network deploy, first-time admin
`docs/12-ghl-live-pull.md`	The 18 Aug 2026 pull results
`docs/13-model-realignment.md`	Why the model changed after the pull
`docs/migration-guide/`	The 33-chapter written guide (client/management artefact)
`out/implementation_status.md`	Per-feature status, generated, not asserted

When this runbook and a detail document disagree, the detail document wins on detail and this one wins on sequence. Fix the disagreement rather than choosing — an out-of-date runbook is how an operator gets hurt.

§1 · The project in plain language

1.1 One paragraph

Aspire's CRM is moving off GoHighLevel onto software Aspire runs itself: **Twenty** — the CRM, holding contacts, companies, pipeline and a data model shaped like Aspire's actual training business — and **n8n**, the automation engine that runs forms, lead routing, webinar reminders, enrolment chasing, email and alerting. Both are open source. Both run on one server. The data lives in Aspire's own Postgres database. The build is scripted end to end: one command goes from an empty machine to a working CRM with automation running and verified.

1.2 Why we are doing it — three reasons, in order

1. GoHighLevel cannot model this business. The live account is a cybersecurity training funnel — webinar or workshop, then nurture, then a meeting with a trainer, then bootcamp enrolment paid through a FastPayDirect link, then a cohort — flattened into contact fields and deal stages. Twenty holds programs, cohorts, enrolments and webinar registrations as real objects, so "who paid for the January bootcamp and who is still sitting on a payment link" is a view, not a spreadsheet export.

2. GoHighLevel hides its own automation history. There is no API that exposes what a workflow did or whether it failed. That gap is what made this audit hard, and it is why every automation we build writes an `automationRun` record back into the CRM. Every run, success and failure, is queryable.

3. Aspire is a security company. Client data sitting in a marketing SaaS is a harder conversation than client data sitting on infrastructure Aspire controls, patches and backs up. We sell compliance; the internal systems should meet the standard we sell.

1.3 What we are not claiming

Honesty here is what makes the rest credible.

- We are **not** claiming Twenty replaces GoHighLevel feature for feature. It cannot send SMS, it cannot make phone calls, and no configuration changes that (§3.3).
- We are **not** claiming the whole surface is built. 37 of 111 catalogued features are working or native today (§18). 33 are deliberately out of scope with a recorded reason. 41 are real remaining work.
- We are **not** claiming the six live GoHighLevel workflows were copied. Their internals are unavailable through any API; they were **rebuilt from surrounding evidence** and must be checked against the GHL UI before cutover (§4.7, `docs/13`).
- We are **not** claiming an untested backup is a backup. Restores are proven on the machine that holds the data, monthly (§13.2).

1.4 The deliverables

Deliverable	Where	State
Feature audit — 111 features, each with a disposition and evidence	`out/feature_audit.csv`	Complete
Measured implementation status	`out/implementation_status.md`	Generated per run
The object model, version-controlled	`reference/twenty_schema.yaml`	31 custom objects
The automation library	`n8n/workflows/*.json`	17 live + 2 dev-only

Deliverable	Where	State
The written migration guide	`docs/migration-guide/`, `out/*.pdf`	33 chapters + Bangla explainer
The running system	The VPS	Deploy rehearsed; see §9
This runbook	`docs/RUNBOOK.md`	You are here

§2 · Scope, decisions and open questions

2.1 In scope

Replacing, on infrastructure Aspire controls: the CRM record layer, lead capture, lead routing and scoring, nurture, webinar operations, bootcamp enrolment operations, trainer booking follow-up, consent enforcement, transactional and 1:1 email, click tracking, scheduled sweeps, alerting, and the execution history GoHighLevel never exposed.

2.2 Out of scope, deliberately

Each of these is a decision with a reason, not a gap:

Out of scope	Reason
SMS, MMS, voice, IVR, call recording, voicemail drop	Requires a licensed carrier connection and A2P registration. **Regulatory, not technical.** GHL evidence shows light use
WhatsApp Business	Meta only issues API access through approved providers
Website and funnel hosting	aspiretss.com stays where it is
Course delivery / memberships	aspireelearning.com already does this. Two truths is worse than one
Migrating contact records as part of the audit	The audit sizes the migration; §11 performs it
Choosing replacement vendors	The audit sizes the decision; it does not make it

2.3 Decisions on record

#	Decision	Consequence
D1	**GoHighLevel is out entirely in Phase 1.** Twenty + n8n carry everything	Every channel feature needs a native, n8n or self-hosted answer — see §3.3 and §18.3
D2	**Build everything achievable with Twenty and n8n alone.** Third-party services deferred, not designed out	~85% of the catalogued surface is buildable now; the rest is concentrated in telephony
D3	**Keep `serviceSubscription`, `complianceEngagement`, `phishingBaseline`** even though the GHL account shows no trace of them	Deleting a provisioned Twenty object is destructive and awkward to reverse. Absence from the marketing system is not absence from the business. They stay provisioned and empty, and **the automation does not pretend they are in use**
D4	**Workflow logic is inferred from evidence, not captured**	The six published GHL workflows were rebuilt from form names, field keys, tag vocabulary, calendar names, trigger links and template names. **Each must be spot-checked against the GHL UI before cutover**
D5	The image tags for Twenty and n8n are **pinned** and stay pinned	`latest` can run migrations you did not choose, at a time you did not choose

To overturn D3: delete those three blocks from `reference/twenty_schema.yaml`, drop `wf_renewal` from `BUILDERS` in `scripts/build_n8n_workflows.py`, and remove the renewal branch from `SYS Daily Health Check`.

2.4 Open questions still outstanding

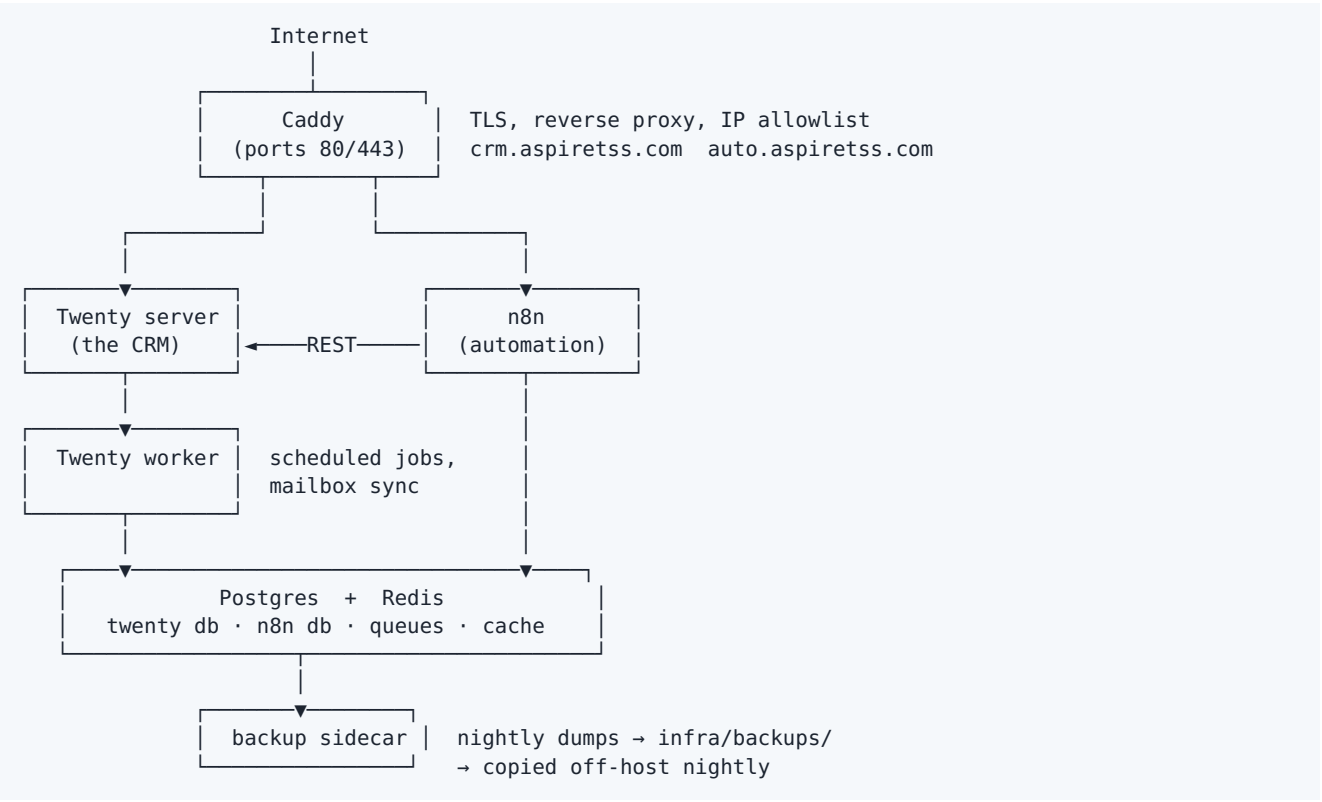
Full text in `docs/01-scope-and-questions.md`. Current state:

#	Question	State
Q1	Is GHL out entirely in Phase 1, or retained for sending?	**Answered — D1.** Evidence from the live pull says dropping GHL entirely means replacing at least three channels
Q2	Agency or sub-account access?	**Answered.** Sub-account. Tier 2 (snapshot diff) is closed; those inventories moved to the manual sweep
Q3	What scopes are on the token?	**Answered** for the 18 Aug token. Three endpoint families returned 401 on missing scopes (§4.6)
Q4	Where does the token live, where does the puller run?	**Answered.** On the operator's machine, `.env` mode `600`, gitignored. The token has since been deleted
Q5	Which evidence window?	**90 days default; 365 for anything named like a renewal, review or annual reset** before dropping it
Q6	Where does workflow execution volume come from?	**Still open.** Not in the public API. UI execution log or the session route. This is why draft/published status is the only structural signal we have
Q7	Does a GHL sunset date exist?	**Still open, and now a contract-termination date.** Gated by the consent export (§11.2), not by engineering
Q8	Denominator for the gap list	**Answered.** All percentages in this runbook are against the **111-feature catalogue**, and say so

Q6 and Q7 are the two that still need a decision from the business. Neither blocks the build; both block the cutover date.

§3 · Architecture

3.1 The shape of it



Seven containers: `caddy`, `server`, `worker`, `n8n`, `db`, `redis`, `backup`. On a laptop, an eighth — `mailpit` — behind a compose profile the server never enables.

3.2 Division of labour

Layer	Does	Does not
Twenty	System of record. Objects, fields, relations, views, roles, mailbox sync, simple record-triggered workflows	Send campaigns, host forms, take bookings, invoice, SMS, telephony
n8n	Orchestration. Public forms, webhooks, schedules, branching, retries, error routing, HTTP to anything, SMTP sending	Store anything durable. It is not a database, and nothing may live only in n8n
Postgres	All state, both applications, separate databases	—
Caddy	TLS termination, hostname routing, and the **editor IP allowlist**	—
The scripts	Build, provision, validate, deploy, prove	Anything against GoHighLevel other than GET

The rule that keeps this coherent: Twenty holds the truth about what happened; n8n makes it happen. If an automation does something a human would later want to query, it writes a record. That is why `automationRun` exists.

3.3 The capability boundary — the hard ceiling

Verified against Twenty's own documentation and the live instance. This is not a configuration gap and no amount of engineering closes it:

Twenty workflow triggers: record created · record updated · record created-or-updated · record deleted · manual · schedule (cron, UTC) · inbound webhook.

Twenty workflow actions: create / update / delete / search / upsert record · iterator · filter · delay · send email · form · code · HTTP request · AI agent (marked coming soon).

The ceiling: Twenty's `Send Email` action sends from a **synced mailbox, one recipient at a time**. Twenty has no campaign engine, no suppression list, no domain warm-up, no deliverability infrastructure, no SMS, no telephony, no public forms, no landing pages, no booking pages, no invoicing or CPQ.

n8n can orchestrate a send. It cannot be the sender.

Three things are therefore genuinely impossible without a third party:

Capability	Why
SMS / MMS	Licensed carrier connection and A2P registration. Regulatory
Voice, numbers, IVR, recording	Same
WhatsApp Business	Meta issues API access only through approved providers

Everything else on the GoHighLevel surface has a path in this plan. Public forms in particular are not on this list — n8n's Form Trigger serves real multi-page forms with validation and file upload at a public URL. An earlier version of the analysis put forms in the vendor column; that was wrong, and correcting it moved a whole cluster back in scope.

3.4 The nevers

Each has an incident behind it. None is a preference.

1. **Never issue anything but GET against GoHighLevel.** The puller refuses non-GET at the transport layer. Do not remove that guard.
2. **Never pull PII.** No contact records, conversation bodies, message content, call recordings or form-submission payloads. Endpoints classed `volume` in the registry have their bodies discarded before anything is written to disk — counts, field names and categorical histograms only.
3. **Never print, log or commit a secret.** `GHL_TOKEN`, `TWENTY_API_KEY`, `N8N_API_KEY`, any session JWT. The puller redacts `Authorization` headers from all error output.
4. **Never commit `raw/`, `workflows/`, `.env`, `infra/.env`, or `.session_curl.txt`.** They contain company data or credentials.
5. **Never remove the rate limit.** 80 requests / 10s against GHL's 100/10s ceiling, exponential backoff on 429, 200,000/day.
6. **Never send a GHL request without `Version: 2021-07-28`.** Omitting it produces an error that reads exactly like an auth failure.
7. **Never run `seed_demo_data.py` on the server.** It is demo data.
8. **Never pass `--dev` to a server deploy.** It deploys the alert sink, which swallows alerts that should reach a real person, and the failure probe, which throws on purpose.
9. **Never mark a row `in_use = yes/no` without evidence.** `unknown` is a legitimate, reportable state. Guessing `no` silently kills a running automation on go-live day and nobody notices for three weeks.

10. **Never regenerate `ENCRYPTION_KEY` against an existing database.** Every secret Twenty holds becomes permanently unreadable. There is no recovery path and no support to call.
11. **Never switch an image tag to `latest`.**
12. **Never skip a gate** because the previous run passed.

---

§4 · Part A — The GoHighLevel audit

Purpose: produce an evidence-backed answer to one question — for every GoHighLevel feature Aspire actually uses, can Twenty + n8n replace it, and if not, what does it cost to replace it another way?

Status: complete. The live pull ran on **18 August 2026** against location `62iXlYxxHYUv14IS2LjG` (Aspire Tech Services and Solution Corp, US, `America/New_York`). The token was deleted afterwards. **This section is the procedure for re-running it**, which will be needed if the account changes materially before cutover.

4.1 Governing principles

1. **Extract with tools, judge with people.** The blueprint is already in the system as structured data. Screenshotting the UI photographs the wall instead of reading the plumbing diagram — 200 workflows × 10 nodes is 2,000 clicks and no way to know what you missed.
2. **Discard before you map.** Every feature retired is a feature nobody has to migrate, test or explain. The usage filter removes a third to a half of the surface. Do it first.
3. **Evidence or `unknown` — never a guess.**
4. **Verify empirically, then automate.** The first puller run is a discovery run: it records status and response shape per endpoint and treats failures as data, not crashes.
5. **Config, never contacts.**

4.2 The six tiers

Tier	What	Automated?
0	Access and scope — agency vs sub-account, create the read-only token, record the granted scope list	Manual, blocks everything
1	Structural pull — 43 endpoints: custom fields, values, tags, pipelines, calendars, forms, surveys, templates, products, invoices, funnels, social, users, workflow names and statuses	`scripts/ghl_pull.py`
2	Snapshot diff — agency only. **Closed for us**; sub-account token	n/a
3	Usage filter — the discard pass. Forms with no submissions, calendars with no bookings, channels with no conversations	Derived from Tier 1
4	Workflow internals — the authenticated-session route	`scripts/ghl_workflow_capture.py`
5	Manual sweep — domains, DNS, A2P, marketplace apps, Zapier/Make, memberships, dashboards	`docs/04-manual-sweep.md`
6	Classification and report	`scripts/build_audit.py`

Each tier is independently useful. If the audit is cut short, the completed tiers still stand.

4.3 Tier 0 — access, token, scopes

1. In GoHighLevel: **Settings → Integrations → Private Integrations → create new.**
2. **Tick every scope ending `.readonly`. Tick nothing implying write.** The exact list varies by account tier — do not force it.
3. Copy the token immediately. It is shown once and never again.

4. **Paste the granted scope list into the audit notes.** That list defines what the audit can reach and belongs in the report as a stated limitation. Any 401/403 in the coverage report maps to a missing scope, not a bug.
5. Check whether **Snapshots** appears in the settings menu. Present → agency access, Tier 2 runs. Absent → sub-account, Tier 2 is closed and the funnel / website / dashboard inventory moves to Tier 5.

> A sub-account Private Integration **cannot discover its own location id**. `/locations/search` returns 403 and `/oauth/installedLocations` is out of scope. Take the location id from the browser URL: `app.gohighlevel.com/v2/location/ <THIS> /settings/...`

Set up the working copy:

```
git clone https://github.com/rafilovestosuffer/aspire-crm-twenty.git
cd aspire-crm-twenty
cp .env.example .env
chmod 600 .env
```

Fill `.env`:

```
GHL_TOKEN=pit-xxxxxxxxxxxxxxxx      # shown once, never again
GHL_LOCATION=62iXlYxxHYUv14IS2LjG   # from the browser URL
GHL_COMPANY_ID=                     # agency id, or blank
TWENTY_BASE_URL=http://localhost:3000
TWENTY_API_KEY=
```

Blank `GHL_COMPANY_ID` means agency-class endpoints are recorded as skipped, not failed — which is the honest distinction.

Optional: `pip install -r scripts/requirements.txt`. The scripts run on the standard library alone without it.

4.4 Verify auth by hand before running anything

```
set -a && source .env && set +a

curl -s "https://services.leadconnectorhq.com/locations/$GHL_LOCATION" \
  -H "Accept: application/json" \
  -H "Authorization: Bearer $GHL_TOKEN" \
  -H "Version: 2021-07-28" \
  -H "User-Agent: aspire-audit/1.0" | head -c 600
```

Gate: this returns your location object.

Two failure modes that both read as auth problems and are not:

Symptom	Cause
An error that looks like an auth failure	The `Version` header is missing. It is mandatory
Cloudflare error **1010**	The default Python-urllib User-Agent. Send a User-Agent. `ghl_pull.py` now identifies itself

4.5 Tier 1 — the structural pull

Always in this order. Never start with the full run.

```
python scripts/ghl_pull.py --dry-run                # 43 endpoints, calls nothing
python scripts/ghl_pull.py --only custom_fields     # verify one endpoint
head -40 raw/custom_fields.json                     # does the shape look right?
python scripts/ghl_pull.py                          # full discovery run
```

Useful flags: `--window-days N` (evidence window, default 90 — see Q5), `--force` (re-pull endpoints already on disk).

Then read the coverage report before drawing any conclusion from `raw/`:

```
column -s, -t out/pull_coverage.csv | less -S
```

Status	Meaning
`ok`	Data retrieved
`error 401/403`	Missing scope. Not a broken script. Note it; it is a stated limitation
`error 404`	Endpoint absent on this account tier. Expected for registry rows marked `unverified`
`skipped`	Agency-level, no company id

> **The most dangerous result in this entire repository is a silent zero.** `/emails/builder` returns its array as `builders`, not `data`. With the wrong `list_key` it answers **HTTP 200 and an empty list** — the first pull recorded zero email templates when there were **14**. It would have gone into the audit as "GHL has no templates" and nobody would have questioned a 200. Check `list_key` against a real response before believing any count of 0.

4.6 What the 18 August 2026 pull found

37 of 43 endpoints returned data. GET only, 50 requests, throttled. No PII.

Area	Count	Note
Custom fields	**53**	**all on `contact`** — zero on opportunity
Custom values	6	Calendar Link, Support Phone/Email, Review Link, Business Hours, Privacy Link
Tags	57	heavily used as workflow state and audience segmentation
Users	19	18 admin, 1 user
Businesses	19	
Pipelines	3	11, 8 and 4 stages
Calendars	8	7 active, 1 inactive; 3 personal. **7 bookings in 90 days**
Forms	**44**	42 submissions in 90 days; 200 all-time across 46 forms; **26 forms have never received one**
Surveys	3	**0** submissions in 90 days
Workflows	22	**6 published, 16 draft**
Trigger links	16	
Email schedules	10	3 sent, 7 draft

Area	Count	Note
Funnels / sites	13	
Products	2	both DIGITAL
Invoices	**126**	all `sent`, all `source: workflow`
Media files	188	
Social accounts	5	2 Google, Facebook, Instagram, LinkedIn
Email builder templates	**14**	the real library, at `/emails/builder`
`/locations/{id}/templates` email + SMS	0	genuinely zero, re-checked with and without a type filter
Campaigns	0	
Payment transactions	**0**	authorised, and empty — **no card revenue runs through GHL**

Pipelines — none maps cleanly to Twenty's defaults (`NEW, SCREENING, MEETING, PROPOSAL, CUSTOMER`, and there is **no** `WON`). All three need an explicit stage mapping before any opportunity migrates. See `reference/ghl_stage_map.yaml`.

- **CloudSecurity-Training** (11): New Lead, Contacted, Qualified, Hot Lead, Awaiting Payment, Enrolled, Referral Opportunity, Lead Nurturing, Trial/Free Offer, Evaluation, Success
- **Cyber Security Training** (8): New Lead, Contacted, Qualified Lead, Hot Lead, Opportunity, Negotiation, Closed-Won, Closed-Lost
- **Marketing Pipeline** (4): New Lead, Contacted, Proposal Sent, Closed

Channel evidence, 90 days, counts only: 20 conversations, all `type: TYPE_PHONE`; last message type 11 email, 4 SMS, 4 live-chat info, 1 call. Small but **not empty** — email, SMS and voice are all live. That is the direct evidence behind the deferred telephony decisions.

Known gaps in that pull:

Endpoint	Result	Meaning
`blog_authors`, `blog_categories`	401	token lacks `blogs.readonly`; `blog_sites` returned 0
`payment_subscriptions`	401	token lacks the scope. `/payments/transactions` **is** authorised and returned 0
`custom_menus`	401	marked unverified in the registry; low value
`snapshots`, `sub_accounts`	skipped	agency token only

Four registry defects the live API exposed, all now fixed in `reference/ghl_endpoints.yaml`:

1. `/emails/builder` → `list_key: builders`, not `data`. **HTTP 200, silent zero.**
2. `/calendars/events` refuses a location-wide query — 422 "Either of userId, calendarId or groupId is required". Must fan out per calendar.
3. `/surveys/` caps `limit` at **50**. 100 is a 422.
4. `/medias/files` requires an undocumented `type` parameter. Omitted, 422.

4.7 Tier 4 — workflow internals, and why they are not available

The load-bearing finding of the whole audit: the HighLevel public API exposes workflow **metadata only** — id, name, status, timestamps. No steps, no actions, no triggers, no conditions, no wait durations, no template references.

Confirmed twice: the documented Workflows resource has a single read endpoint, and a standing feature request states plainly that "the platform API is read-only for workflows... There are no write endpoints available." A related request has sat **"under review" since August 2022**.

So `GET /workflows/` answers how many workflows exist and how many are published. It cannot answer what any of them do — which is the entire point of the automation inventory.

The route that works is the authenticated-session route: capture one workflow's step-tree XHR from DevTools and replay it across the workflow list. This uses HighLevel's internal backend, not the public API.

```
python scripts/ghl_workflow_capture.py --dry-run
```

Needs `.session_curl.txt` first:

1. Open any workflow in GHL.
2. DevTools (F12) → Network → XHR.
3. Reload. Find the response containing the **step tree** — not the workflow list.
4. Right-click → Copy → **Copy as cURL (bash)**.
5. Save into `.session_curl.txt` (gitignored).

```
python scripts/ghl_workflow_capture.py --limit 5        # verify the shape first
python scripts/ghl_workflow_capture.py --published-only
```

Open one file in `workflows/` and confirm the steps are genuinely there. **A response that parses is not necessarily the response you wanted.**

Three constraints to plan for rather than discover mid-run:

Constraint	Effect	Handling
Session JWT, not a token	Expires in hours	The script checkpoints per workflow and resumes. **Expect to re-capture once or twice per full run. This is normal, not a failure**
Undocumented, unsupported	Shape can change without notice	Raw responses stored verbatim; parsed in a second pass
Outside the read-only token scopes	Sits outside the token governance story	Disclose it in the writeup rather than leaving it implicit

Legitimate for a one-off internal audit of our own account's configuration. Not a basis for anything ongoing.

This route has not been run. Decision D4 stands: the six published workflows were rebuilt from surrounding evidence. Here is what each was inferred from, so it can be checked against the GHL UI in minutes:

GHL workflow (published)	Rebuilt as	Inferred from
Bootcamp Payment Automation	`ENR Bootcamp Enrolment`	6 FastPayDirect tier links, 126 workflow-issued invoices, `CST-USA-2026` invoice template, `Bootcamp-January-26` form
Training Automation Follow-up	`LEAD Nurture Sequence` + `EVT Post-Webinar Follow-up`	`Training-*` forms, `training-interested` / `watched training webinar` tags
1-on-1 Consultation	`BOOK Trainer Appointment`	`1-on-1 Cyber Security Career Consultation` and 7 other calendars, `booked an appointment` tag
FrontDesk AI - Post Conversation Router	`AI FrontDesk Handover`	the 7 `fdai_*` contact fields
UAE Email Test	not rebuilt	a test; the `dubai` tag is the only trace
n8n workflow	not rebuilt	already n8n's job

The 16 drafts were never published — 8 are named `New Workflow : <timestamp>`. They are DROP candidates on structural grounds. **Confirm in the UI before deleting anything**, because `status` is the only signal the API gives.

4.8 Tier 5 — the manual sweep

Work `docs/04-manual-sweep.md` into `raw/manual_sweep.md`. Half a day. **Copy page text, not screenshots** — text is searchable, diffable and pasteable; an image is none of those.

Record "checked — nothing configured" explicitly. An unticked box means nobody looked, which is a completely different fact from "nothing there", and only one of them is safe.

The two highest-value questions in the sweep:

- **The suppression list — its size, and that the export path actually works.** Not that it exists. That it works. Before any termination date is agreed.
- **Zapier / Make connections touching GHL.** Ask teammates directly. These usually sit in a personal account and are invisible from inside GHL. Absence of evidence is not evidence of absence.

4.9 Tier 6 — build the report

```
python scripts/build_audit.py          # → out/feature_audit.csv
python scripts/build_audit.py --min-records 1
```

Disposition values: `TWENTY` · `N8N` · `TWENTY+N8N` · `DROP` · `ESCALATE` · `UNKNOWN`.

`ESCALATE` means it cannot be done by Twenty or n8n alone and needs a vendor/budget decision. **Do not hide these.** A short, honest ESCALATE list is what makes the rest of the report credible; a report with no ESCALATE items is one nobody should believe.

`criticality` and `effort_days` stay **blank** for a person to fill. They are judgement, not data, and a script that invented them would be lying convincingly.

Gate — and this one is not optional: open GoHighLevel and check **five rows by hand**. If all five are right, trust the rest. If any is wrong, fix the rule and re-run the whole thing. A classification that is 90% right is indistinguishable from one that is 100% right until go-live day.

4.10 What the audit cannot tell you

State these plainly rather than letting them be discovered later:

- Whether a workflow that runs is a workflow that matters. Execution counts measure activity, not value. A renewal escalation that fires four times a year is critical.
- Twenty's performance at Aspire's real data volume. Local Docker cannot establish this.
- Backup/restore behaviour under load, and TLS/proxy behaviour in production, from a laptop.
- Anything living in a teammate's personal Zapier or Make account.

4.11 Closing the audit down

The token was written to gitignored `.env` (mode `600`) and never printed, logged or committed. `raw/` is gitignored and did not survive the session.

Delete the Private Integration in GoHighLevel when the audit is finished. Nothing in this repository depends on it staying alive, and everything the public API can reach has already been pulled. The remaining unknowns — workflow internals and the suppression list — are **not reachable with a Private Integration token at all**, so keeping one alive buys nothing.

---

§5 · Prerequisites and environment

5.1 Laptop (demo / development)

Docker	Docker Desktop, or Docker Engine + Compose v2
RAM	**6 GB free.** Twenty alone wants 4 GB; Postgres, Redis and n8n share the rest
Disk	20 GB free
Python	3.9+
`openssl`	Already present on macOS and Linux
Git	Any recent version

On Windows, run everything from WSL. The most common first failure on a Windows laptop is Docker Desktop's WSL integration being switched off — `up.sh` names that specifically, because every later error is a red herring otherwise.

5.2 Server

CPU / RAM / disk	**4 cores, 8 GB RAM, 60 GB SSD minimum.** 8 cores / 16 GB for real load
OS	Any current Linux with Docker Engine and Compose v2
DNS	Two A records pointing at the server, **already propagated**
Ports	80 and 443 inbound. Nothing else needs to be
Access	SSH with key auth. Password auth off

Two names, both pointing at the same server:

```
crm.aspiretss.com     →  <server ip>     the CRM
auto.aspiretss.com    →  <server ip>     forms, webhooks, automation editor
```

On Contabo's cheaper plans, disk is generous and RAM is thin. This stack wants 8 GB. On 4 GB, Twenty and Postgres contend and the UI feels slow under any real load.

5.3 Things only Aspire can supply

Roughly a third of the remaining work is blocked on one of these. Collect them before a deploy evening, not during one:

Item	Where it comes from	Without it
DNS records	Domain registrar / Contabo panel	Let's Encrypt cannot issue; browsers refuse the site
SMTP user + app password	Google Workspace admin	The deploy **stops** rather than half-working
Office public IP range	`curl ifconfig.me` **run from the office**, not the server	The reverse proxy refuses to start

Item	Where it comes from	Without it
Chat webhook URL	Google Chat space → Apps & integrations → Webhooks. Two minutes	No failure ever reaches a person
Six FastPayDirect payment links	GHL Marketing → Trigger Links, "Payment Link - Training \<Tier>"	**Every bootcamp enrolment returns 500**
Google OAuth client	Google Cloud console	No mailbox sync, no calendar on the timeline
GHL suppression list export	GoHighLevel, before termination	**Unrecoverable.** Compliance incident waiting to happen

5.4 The environment file

There are **two** environment files and they are not interchangeable:

File	Used by	Contains
`.env` (repo root)	The audit scripts	`GHL_TOKEN`, `GHL_LOCATION`, `GHL_COMPANY_ID`, `TWENTY_BASE_URL`, `TWENTY_API_KEY`
`infra/.env`	The stack, and every build/deploy script	Everything below

Both are gitignored. Both must stay that way.

Secrets — generated, never typed:

Key	Notes
`PG_DATABASE_PASSWORD`	Baked into the Postgres volume on first boot. Changing it later means restoring the original or destroying the volume
`ENCRYPTION_KEY`	**Back this up off-host.** Lose it and every secret Twenty holds — OAuth tokens, connected mailboxes, application variables — is permanently unreadable
`FALLBACK_ENCRYPTION_KEY`	Rotation support
`APP_SECRET`	Session signing
`N8N_ENCRYPTION_KEY`	**Back this up off-host.** Every stored n8n credential depends on it

Addressing:

Key	Laptop	Server
`SERVER_URL`	`http://localhost:3000`	`https://crm.aspiretss.com`
`N8N_HOST`	`localhost`	`auto.aspiretss.com`
`N8N_PROTOCOL`	`http`	`https`
`N8N_PUBLIC_URL`	`http://localhost:5678`	`https://auto.aspiretss.com`
`CRM_DOMAIN` / `AUTOMATION_DOMAIN`	—	the two public names
`ACME_EMAIL`	—	for Let's Encrypt notices

`SERVER_URL` must match how people actually reach the CRM, **scheme included**. A mismatch produces a login redirect loop that looks like an auth bug, and it is also what breaks the metadata API (§14.2).

Mail:

Key	Notes
`EMAIL_DRIVER`, `EMAIL_SMTP_HOST/PORT/USER/PASSWORD/SECURE`	Blank host on a laptop points at Mailpit
`EMAIL_FROM_ADDRESS`, `EMAIL_FROM_NAME`, `ASPIRE_FROM_EMAIL`	Gmail rejects a From that is not the authenticated account or an allowed send-as alias
`LIVE_MAIL_ALLOWLIST`	The second gate inside `VEND Send Email`, next to consent. **Empty = production**, consent only
`LIVE_DEMO_EMAIL`	The single address the live-mail proofs are permitted to use

`smtp-relay.gmail.com` is **Workspace-only**; a personal `@gmail.com` must use `smtp.gmail.com`. Port 587 + STARTTLS is what n8n's SMTP node expects.

Operational:

Key	Notes
`TAG`, `N8N_TAG`	Pinned image tags. Twenty is pinned to **v2.1.0**, the first release marked production-ready for single-tenant self-hosting
`ALERT_WEBHOOK_URL`	Where `SYS Error Handler` shouts
`SALES_WEBHOOK_URL`	Where new-lead notifications land
`N8N_EDITOR_ALLOWED_IPS`	Space-separated CIDR. **Unset, the proxy refuses to start**
`ENROL_PAYMENT_LINKS`	`BRONZE=…,SILVER=…,GOLD=…,PLATINUM=…,DIAMOND=…,TITANIUM=…`
`ASPIRE_REP_IDS`	Round-robin assignment targets
`TZ`	**Leave as UTC unless you have read §15.3.** Setting `America/New_York` here is a live trap
`TWENTY_API_KEY`, `N8N_API_KEY`	Written back automatically by the bootstrap scripts

§6 · The object model

31 custom objects, 198 fields and relations, provisioned by `scripts/twenty_provision.py` from `reference/twenty_schema.yaml` — scripted and committed rather than clicked through the UI, so the whole workspace rebuilds in one command. That will happen more than once.

6.1 The idea in one line

Twenty becomes the system of record for every GoHighLevel domain, including the ones it cannot execute. n8n plus (where unavoidable) a vendor does the sending, booking and billing; Twenty holds the truth about what happened.

The alternative — leaving channel data in whatever vendor sends it — recreates the exact problem this migration exists to solve: business truth scattered across systems that do not agree with each other.

Anyone who reads this model as "GoHighLevel replaced" has misread it. The honest framing: the record layer is solved and buildable today; the execution layer is either n8n, native, or procurement.

6.2 The objects that carry the business

The training funnel, added after the 18 August pull showed the original model was aimed at the wrong business (`docs/13`):

Object	Holds
`trainingProgram`	The certification catalogue — SC-200, SC-100, AZ-900, SC-900, Splunk Core Certified User / Power User / Advanced Power User, GCP Cloud Digital Leader
`cohort`	A dated intake with seats, status and enrolments
`enrollment`	One student in one cohort, with payment state
`webinarEvent`	The event at the top of the funnel
`webinarRegistration`	Who registered, who attended, which reminders were sent
`aiConversation`	One record per front-desk AI conversation, instead of seven contact fields that overwrote each other

Two more carry the discipline:

`consentRecord` — the most important object here. Per-person, per-channel consent state with source and proof. GoHighLevel was deliberately authoritative for consent; removing GHL removes that home, and nothing else in the plan replaces it. Three rules must hold or it is decoration:

1. It is populated from the GHL suppression export **before** the account is terminated. After termination that data is gone.
2. **Every** send workflow checks it first. A consent record nothing consults provides no protection and false assurance — worse than none.
3. Vendor-side and inbound unsubscribes write back into it, or it drifts out of date within weeks. That is what `MSG Inbound Events` is for.

`automationRun` — solves this migration's hardest problem, permanently. The GHL public API refuses to expose workflow execution history, which is exactly why the discard pass needed manual UI work. Every n8n workflow here writes its run into the CRM, so the next person auditing this stack queries it in one call. It costs one HTTP node per workflow.

6.3 The full mapping — GHL feature → Twenty object → who executes

GHL feature	Twenty object	Executed by
Conversations / unified inbox	`messageLog`	Mailbox sync + n8n
Call recording & reporting	`callLog`	Deferred — carrier
Email templates & builder	`emailTemplate`	n8n renders; HTML referenced
Campaigns / bulk email	`campaign`	n8n SMTP, throttled, consent-checked
Trigger links	`trackedLink`	n8n serves the redirect
Forms & surveys	`formDefinition`, `formSubmission`	**n8n Form Trigger**
Calendars / booking pages	`appointment`	Cal.com (Phase 3)
Products	`product`	native
Estimates / proposals	`quote`, `quoteLineItem`	DocuSeal (Phase 3)
Invoices	`invoice`	Finance system
Payments / orders	`payment`	FastPayDirect, off-platform
Funnels / websites / blogs	`landingPage`	Out of scope — stays where it is
Social planner	`socialPost`	Deferred
Reputation / reviews	`review`	Deferred
Memberships / courses	`membershipEnrollment`	aspireelearning.com
Tags	`crmTag`	native + `SEG Tag Sync`
Custom values	`mergeVariable`	native
DND / suppression	`consentRecord`	**native, enforced by n8n at send time**
Workflow execution history	`automationRun`	n8n writes back

Plus the three kept-but-empty service objects from D3: `serviceSubscription`, `complianceEngagement`, `phishingBaseline`.

6.4 Provisioning it

```
python scripts/twenty_provision.py --dry-run              # prints every payload, sends nothing
python scripts/twenty_provision.py --only consentRecord   # one object first
python scripts/twenty_provision.py                        # full provision
python scripts/twenty_provision.py --objects-only         # skip fields and relations
python scripts/twenty_provision.py --skip-relations       # scalar fields only
```

Idempotent — existing objects and fields are skipped, never duplicated, so a partial run is fixed by running it again. Three passes (objects → fields → relations), because relations cannot be created before both ends exist. Report lands in `out/twenty_provision_report.json`.

If it fails: metadata payload shapes vary between Twenty versions. Failures print the API's own error text rather than a wrapped one. The likeliest divergence is relation creation — `relationCreationPayload` is the current shape, older versions used a separate endpoint. Use `--skip-relations` to get everything else in; scalar fields are the bulk of the value and nothing depends on relations landing first.

If it fails with HTTP 500 and an empty body, it is not authentication. See §14.2 — that is the metadata self-call, and it is the single most misleading failure in this system.

Verify:

```
python scripts/twenty_probe.py            # → out/twenty_capability.md
python scripts/twenty_probe.py --expect trainingProgram,cohort,enrollment
```

Gate: the probe reports the training objects present.

6.5 What a person still has to decide

- **Volume on `messageLog`.** Logging every marketing email at real send volume will dwarf every other object. Log 1:1 and transactional messages individually; keep campaign sends aggregated on `campaign`.
- **Whether `membershipEnrollment` is needed at all.** If A-SAT already runs on aspireelearning.com, this is duplicate structure — drop it rather than maintain two truths.
- **`crmTag.migrationAction`.** Every one of the 57 GHL tags needs a decision: keep as a tag, convert to a field, or drop. Tags encoding workflow state should become fields. `SEG Tag Sync` proposes; a person confirms.

---

§7 · The automation library

19 workflow files: 17 live, 2 dev-only. Generated by `scripts/build_n8n_workflows.py`, committed as JSON in `n8n/workflows/`, deployed by `scripts/n8n_deploy.py`.

Do not edit the JSON by hand and do not author in the n8n UI. The generator is the source of truth; it refuses to emit code that would fail at runtime, and the deploy script binds sub-workflow and error-workflow references by id. An edit made in the editor is lost on the next deploy and unreviewable in between.

```
python scripts/build_n8n_workflows.py --list      # what it builds
python scripts/build_n8n_workflows.py             # regenerate all
```

7.1 The seventeen

Workflow	Trigger	What it does
`LEAD Form Intake`	Public form	Dedupe → person, and a company **only when the address is not a personal one** → consent → score on intent → route to a queue with an SLA → task → acknowledgement → sales notification
`VEND Send Email`	Sub-workflow	**The only path to SMTP.** Checks consent, checks the allowlist, renders the template, logs a `messageLog`. No other workflow may send directly
`MSG Tracked Link Redirect`	Webhook	Counts the click, then 307s to the destination
`MSG Inbound Events`	Webhook	Closes the consent loop: an unsubscribe or bounce flips the consent record, and a later form submission cannot quietly undo it
`LEAD Nurture Sequence`	Schedule	Day 3, 7 and 14 after an enquiry, then stops. Stops early on a reply, an opt-out, or a real opportunity on that person's company
`EVT Webinar Registration`	Webhook	Person, registration, consent, confirmation — the top of the funnel
`EVT Webinar Reminders`	Hourly	The 24-hour and 1-hour reminders, **counted on the registration** so a re-run cannot send twice
`EVT Post-Webinar Follow-up`	Daily	Recording to the absent, survey to the present, attendance written back onto the event
`ENR Bootcamp Enrolment`	Webhook	Creates the enrolment as `PAYMENT_SENT`, issues the tier's payment link, raises a task. **Never marks anything paid**
`ENR Cohort Operations`	Daily	Chases unpaid enrolments every third day, sends joining instructions, recomputes seat counts from the enrolments themselves
`BOOK Trainer Appointment`	Daily	Reminds about tomorrow's meetings across the eight calendars, flags the ones nobody attended
`AI FrontDesk Handover`	Webhook	One record per AI conversation, and a task when it asks for a human
`SEG Tag Sync`	Daily	Classifies each tag: stay a tag, become a field, or be dropped
`SUB Renewal Escalation`	Daily	90/60/30/7 days before renewal. **Dormant while no subscriptions exist, and says so** rather than reporting success over an empty table

Workflow	Trigger	What it does
`OPS Scheduled Sweeps`	Daily	Finds what did not happen — stale opportunities, no-shows, overdue invoices
`SYS Error Handler`	Error trigger	Catches any failure, records it, alerts, suppresses alert storms
`SYS Daily Health Check`	Daily	Asks whether the seven scheduled jobs ran in the last 26 hours — **each queried by name**, not read off the latest page of `automationRun`. Does not check backup freshness; that is `verify_restore.py`

7.2 The two dev-only workflows

`SYS Alert Sink (dev)` and `SYS Failure Probe (dev)` deploy **only** with `--dev`. The sink swallows alerts that must reach a real person; the probe throws on purpose. `n8n_deploy.py` refuses to deploy either without the flag, so a production alert can never be swallowed and nothing in production throws deliberately.

The probe is **webhook**-triggered, not manual, for a specific reason: `settings.errorWorkflow` fires **only for production executions**. A manual run fails in the editor and the error workflow is never called — so testing failure handling by hand proves nothing.

7.3 Design rules every workflow follows

1. **One send path.** Everything routes through `VEND Send Email`. Consent and allowlist are enforced once, in one place, with proof.
2. **Every run writes an `automationRun`.** Success and failure both.
3. **Every scan pages.** A Twenty page is 200 rows and nothing raises when you ask for more (§15.2).
4. **Every scheduled workflow distinguishes "nothing due" from "nothing exists".** Reporting success over an empty table is how a dead automation looks healthy for months.
5. **Idempotence where a re-run could double-send.** Reminder counters live on the record, not in the workflow.
6. **No credential inline in a node, ever.** Always the n8n credential store.
7. **Money is never asserted by an automation.** `ENR Bootcamp Enrolment` issues a payment link and stops. A human runs `mark_enrolment_paid.py` after FastPayDirect clears. The CRM does not pretend a link was a receipt.

7.4 Deploying them

```
python scripts/n8n_credentials.py --dry-run
python scripts/n8n_credentials.py                # Twenty header auth + Aspire SMTP
python scripts/validate_workflow_queries.py      # every Twenty call vs the LIVE schema
python scripts/validate_workflow_queries.py --offline   # weaker: schema file, no stack
python scripts/n8n_deploy.py --dry-run
python scripts/n8n_deploy.py --dev --activate    # laptop
python scripts/n8n_deploy.py --activate          # server — no --dev, ever
python scripts/n8n_deploy.py --only lead-form-intake
```

`n8n_credentials.py` writes the `Twenty API` header-auth credential and an `Aspire SMTP` credential. With the SMTP block blank it points at Mailpit, so the send path is fully exercisable with no mail account and no chance of reaching a real contact.

Sub-workflow and error-workflow references **resolve by id, not name**. Named, they dangle silently — every workflow looks like it has error handling and none of it fires. `n8n_deploy.py` binds the ids at deploy time. **This is why you deploy through the script and never by importing JSON in the UI.**

---

§8 · Part B — Building the stack on a laptop

From an empty machine to a CRM you can demo. About **12 minutes** for the scripted path, most of it Twenty's first-boot migrations.

8.1 The whole thing in one command

```
git clone https://github.com/rafilovestosuffer/aspire-crm-twenty.git
cd aspire-crm-twenty
./infra/rebuild.sh
```

Eleven gated steps, stopping at the first failure:

#	Step	Command it runs
1	Stack up, secrets generated	`infra/up.sh`
2	Verify every layer	`stack_verify.py`
3	Create the Twenty workspace	`bootstrap_workspace.py`
4	Create the n8n owner + API key	`bootstrap_n8n.py`
5	Provision the object model	`twenty_provision.py`
6	Seed demo data	`seed_demo_data.py --wipe`
7	Create n8n credentials	`n8n_credentials.py`
8	Validate every Twenty call	`validate_workflow_queries.py`
9	Deploy and activate	`n8n_deploy.py --dev --activate`
10	Run the workflows and assert what happened	`prove_workflows.py`
11	Re-verify every layer	`stack_verify.py`

`./infra/rebuild.sh --destroy` wipes the databases first.

Step 6 passes `--wipe` deliberately: without it, a second run of the same script accumulates a second copy of the demo data.

The rest of this section is **what each step does and why**, for when one of them fails.

8.2 Step 1 — start the stack

```
./infra/up.sh               # start; generates any missing secrets
./infra/up.sh --no-dev      # start without the local mail catcher
./infra/up.sh --vps         # start with the VPS overlay (loopback binding)
./infra/up.sh --down        # stop, keep data
./infra/up.sh --destroy     # stop and DELETE ALL DATA
```

`up.sh` copies `infra/.env.example` to `infra/.env`, fills any **blank** secret, pulls images, starts everything and waits for Twenty to report healthy. It never overwrites a secret that already has a value — regenerating `ENCRYPTION_KEY` against an existing database makes every stored secret unreadable.

> **Back up `ENCRYPTION_KEY` and `N8N_ENCRYPTION_KEY` from `infra/.env` to somewhere other than this laptop, now.** There is no recovery path.

First boot runs Twenty's migrations. Give it five minutes before concluding anything.

8.3 Step 2 — verify every layer

```
python scripts/stack_verify.py
python scripts/stack_verify.py --verbose
```

Checks Postgres, **both** databases, Redis, Twenty's health endpoint, **the worker container separately**, n8n, and both APIs.

The worker gets its own check because **Twenty's UI looks completely healthy while the worker is dead** — and a dead worker means no scheduled workflows and no mailbox sync, silently. This is the single most under-appreciated failure mode in the stack.

Gate: all green.

8.4 Steps 3–4 — workspace and API keys

```
python scripts/bootstrap_workspace.py     # Twenty user, workspace, API key
python scripts/bootstrap_workspace.py --show
python scripts/bootstrap_n8n.py           # n8n owner + API key
```

Neither needs a browser, so a rebuild is reproducible rather than a sequence of remembered clicks. Each writes its key back into `infra/.env`. By hand instead: Twenty → Settings → API & Webhooks, and n8n → Settings → n8n API.

Laptop login for both: `admin@aspiretss.com` / `AspireDemo2026!` — **change it before the machine is on a network anyone else can reach.** It is in a public repository.

8.5 Step 6 — seed the demo data

```
python scripts/seed_demo_data.py --dry-run
python scripts/seed_demo_data.py            # ~6 minutes
python scripts/seed_demo_data.py --wipe     # reset, then seed
```

448 records, deterministic — every run produces identical records, so the demo looks the same at rehearsal and on the day.

The data is shaped for the demo on purpose:

Detail	Why
8 subscriptions renewing within 90 days	The escalation workflow has real targets
8 contacts opted out	Lets you show the consent gate refusing a send, not just describe it
7 training accounts above 90% seat usage	A visible upsell signal
Phishing scores trending down over three quarters	The account-health story in one chart

`--wipe` **deletes only seeded data**, identified by the `.example.com` addresses (RFC 2606), the `[demo]` task prefix, and any foreign key pointing at a seeded parent — company, person **or training account**. Matching on `companyId` alone misses consent records and phishing baselines, because those hang off a person and a training account; they would then accumulate on every reseed. With no seeded parent found it deletes nothing, by design.

Gate: no view shows an empty state.

8.6 Step 5 revisited, and native Twenty configuration

Provisioning is §6.4. Then the configuration-only work, which is a large part of the demo and needs no n8n:

Saved views, one per demo beat:

View	Object	Filter
Open for enrolment	Cohort	status open, sorted by start date
Unpaid enrolments	Enrollment	`paymentState = PAYMENT_SENT`
Webinar registrations — next event	Webinar Registration	by event
Pipeline	Opportunity	Kanban by stage
Renewals — next 90 days	Service Subscription	`renewalDate` within 90 days (dormant until the service line is real)

Two native Twenty workflows, so automation visibly lives in the CRM too — for example Opportunity stage → Proposal creates a follow-up task.

One non-admin role with least privilege, to show it is not a toy.

Email + calendar sync. Settings → Accounts → Connect Gmail. Gmail / Workspace and Microsoft sync two-way; IMAP and CalDAV also work. Restrict which labels or folders sync and exclude shared addresses (`support@`, `info@`). Threads and calendar events then appear on record timelines, resyncing every ~5 minutes. **This is the honest answer to "where did the GHL inbox go?" for email** — which is most of Aspire's conversation volume.

Mailbox sync is **not** the same thing as workflow sending. Workflow mail is n8n SMTP via the `Aspire SMTP` credential. Both can use the same account; they are configured separately and prove different things.

8.7 Steps 8–10 — the three gates, and why each exists

```
python scripts/build_n8n_workflows.py         # gate 1
python scripts/validate_workflow_queries.py   # gate 2
python scripts/prove_workflows.py             # gate 3
```

Gate 1 — the generator refuses to emit broken code. Chiefly a nested `}}`, which closes an n8n expression early and turns the remainder into a syntax error. Also: `jsonBody` only evaluated when the whole parameter starts with `=`; `$env` rather than `process.env` in Code nodes; a Switch `fallbackOutput` that must be an existing index or `"extra"`; `.toUTC().toISO()` on every date; and `encodeURIComponent` on every interpolated filter value.

Gate 2 — every Twenty REST call, checked against the running instance. This is the important one. Twenty does not reliably reject a malformed query: send the `filter[field][eq]=value` shape that n8n users reach for first and it **ignores the filter and returns the whole table**. Nothing errors. The

workflow uses row 0 and carries on. That defect sent a lead acknowledgement to an unrelated contact here, and the only visible symptom was the wrong first name in the email.

`--offline` checks against `reference/twenty_schema.yaml` instead of a live instance. It cannot see drift or anything provisioned by hand, and it says so in its own output rather than claiming a live pass. Use it when there is no stack; never as the final gate.

Gate 3 — prove it did the thing. `prove_workflows.py` does not check that workflows exist. It submits the real form, follows the real redirect, breaks something on purpose, triggers the real schedules, and then asks the CRM whether the records are there. **25 checks.**

Proves	Check
Lead capture	Person, company, linked consent, scored task, acknowledgement **to the right address**
Consent enforcement	A non-consenting contact → send **refused**, no mail, refusal logged with its reason
Click tracking	307 to the destination, `clickCount` incremented
Failure handling	A deliberate failure produces `automationRun` FAILED carrying the real error, and a delivered alert. **Laptop `--dev` only**
Scheduled work	Renewals, sweeps and nurture start and finish without node errors
Health check	`SYS Daily Health Check` runs and writes an `automationRun`

```
python scripts/prove_workflows.py --only consent
python scripts/prove_workflows.py --keep              # leave the proof records behind
python scripts/prove_workflows.py --live-email you@gmail.com
python scripts/prove_workflows.py --production --live-email you@...
python scripts/prove_paging.py                        # pushes a table past 200 rows
```

If `EMAIL_SMTP_HOST` is a real relay, `prove_workflows.py` **refuses** unless `--live-email` (or `LIVE_DEMO_EMAIL`) is on `LIVE_MAIL_ALLOWLIST` — otherwise a rebuild would SMTP the fake proof domains it submits (`proof.{tag}@northgate-{tag}.com`). `rebuild.sh` passes the flag from env, or fails with the same message.

8.8 What a passing suite still does not prove

A check only proves the code it actually executed. These defects passed every check before they were found, and they are the pattern to watch for:

- **The nurture sequence asked for email templates that did not exist.** On a fresh database no lead is old enough to be nudged, so the send path never ran and the suite went green over code that would have thrown on every send.
- **Every scheduled scan asked Twenty for 500 records.** Twenty's ceiling is **200** and it returns 200 with HTTP 200 and no warning. With 32 form submissions in the test data, nothing noticed. The workflows now page, and `prove_paging.py` is the only check that can catch this.
- **The daily health check read the latest 20 `automationRun`s** and looked for two schedule names in that page. Form intake writes a run on every submit, so the schedules fell off the page — it reported a dead worker on a healthy stack, and would have missed a genuinely failed sweep. It now queries each job by name and time.

All three were **volume-dependent**: correct on a demo dataset, broken in production some months in. When adding a check, ask what data would have to exist for it to execute at all.

8.9 Where things are, on a laptop

CRM	http://localhost:3000
Automation	http://localhost:5678
Mail catcher (Mailpit)	http://localhost:8025
Public form	http://localhost:5678/form/aspire-contact

Nothing leaves the machine. Email goes to the local catcher, so the send path runs for real with no mail account and no chance of reaching a live contact.

8.10 The live Gmail demo (optional, laptop only)

Run this **on the laptop that already hosts the stack**. An app password in git or in a remote VM is how credentials leak.

Part 1 — n8n SMTP, proving the workflows really send.

1. Google account → 2-Step Verification → App Password for Mail.
2. In `infra/.env`:

```
EMAIL_DRIVER=smtp
EMAIL_SMTP_HOST=smtp.gmail.com
EMAIL_SMTP_PORT=587
EMAIL_SMTP_USER=you@gmail.com
EMAIL_SMTP_PASSWORD=<app password>
EMAIL_SMTP_SECURE=false
EMAIL_FROM_ADDRESS=you@gmail.com
ASPIRE_FROM_EMAIL=you@gmail.com
LIVE_DEMO_EMAIL=you@gmail.com
LIVE_MAIL_ALLOWLIST=you@gmail.com
```

1. Recreate n8n so the allowlist reaches the container, then rebuild the credential and the send workflow:

```
docker compose -f infra/docker-compose.yml up -d n8n
python scripts/n8n_credentials.py          # Aspire SMTP → smtp.gmail.com
python scripts/build_n8n_workflows.py      # emit the allowlist gate
python scripts/n8n_deploy.py --activate    # not --dev: do not swallow alerts
python scripts/prove_live_mail.py
python scripts/prove_live_mail.py --nurture
python scripts/prove_live_mail.py --cleanup
```

`prove_live_mail.py` refuses unless SMTP is a real relay **and** the recipient is on the allowlist. It upserts you, submits the real form, asserts a `messageLog` `SENT` for `smtp/lead_ack`, posts an unsubscribe to `/webhook/mail-event`, submits again and asserts `Blocked:` on the person, optionally backdates the submission by three days to trigger the nurture, then runs renewal and sweeps.

A Gmail address on the lead form scores **-40** (free domain). The acknowledgement still sends; the lead just looks cold. Use a Workspace address for the high-score demo beat.

Part 2 — Twenty mailbox sync, putting the inbox on the record.

Google Cloud (Testing mode is enough for one user):

1. New project → enable **Gmail API**, **Google Calendar API**, **People API**.
2. OAuth consent screen: External, Testing, add yourself as a test user.

3. OAuth client type **Web application**.
4. Authorized JavaScript origin: `http://localhost:3000`.
5. Redirect URIs — must match `SERVER_URL` **byte for byte, including the scheme**: - `http://localhost:3000/auth/google/redirect` - `http://localhost:3000/auth/google-apis/get-access-token`

```
MESSAGING_PROVIDER_GMAIL_ENABLED=true
CALENDAR_PROVIDER_GOOGLE_ENABLED=true
AUTH_GOOGLE_ENABLED=true
AUTH_GOOGLE_CLIENT_ID=...
AUTH_GOOGLE_CLIENT_SECRET=...
AUTH_GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/redirect
AUTH_GOOGLE_APIS_CALLBACK_URL=http://localhost:3000/auth/google-apis/get-access-token
```

Recreate server and worker. Twenty defaults to config-in-database; if the Admin Panel disagrees with `.env`, set the values under **Settings → Admin Panel → Configuration Variables** as well.

Then confirm the messaging crons are registered — the worker has `DISABLE_CRON_JOBS_REGISTRATION=true` and the server registers the default set, so **verify rather than assume**:

```
docker compose -f infra/docker-compose.yml exec worker \
  yarn command:prod cron:messaging:message-list-fetch
# also messages-import, calendar-event-list-fetch, calendar-events-import
```

This proves: consent gate, template render, SMTP, `messageLog`, form → person → acknowledgement in a real inbox, unsubscribe honoured, nurture send, error alerts if a chat webhook is set, and the mailbox thread on the record.

This does not prove: bounce/complaint webhooks (SMTP has none), bulk or campaign sending, deliverability to third parties, or anything about GHL workflow internals.

8.11 The demo — ten minutes

Order matters. Lead with the data model, land on the live form.

#	Show	Say
1	A company record: people, a webinar registration, an enrolment, a cohort	"GoHighLevel flattens the training funnel into contact fields and a deal."
2	Open for enrolment on a cohort, unpaid vs paid	"Who is still on a payment link — one click."
3	A contact timeline with a real email thread and calendar event	"This is where the inbox went for email. Native, syncs every five minutes."
4	**Fill the public form live on screen**	"Watch." Person appears scored and assigned, task raised, acknowledgement sent. **"That form is hosted by our own stack. No third-party tool."**
5	Try to send to an opted-out contact	It refuses and logs why. "This is the control we tell clients to implement."
6	Mark an enrolment paid, watch joining instructions go	"Payment is off-platform. The CRM does not pretend a link was a receipt."
7	The `automationRun` view	"Every run, success and failure. GoHighLevel exposes no execution history at all — that gap is what made the audit hard."

Watching a record appear from a form submission does more than any document.

---

§9 · Part C — Deploying to the company server

The laptop stack and the server stack are **the same files**. What changes is `infra/.env`, one compose overlay, and Caddy in front. Nothing is rebuilt and nothing is untested at cutover.

Budget half a day, most of it waiting for DNS.

> This section is for a server with a **public IPv4 address** — which is what makes real certificates, working public forms and inbound webhooks possible. If someone calls it "the office server", that is about who pays for it, not where it sits on the network. Confirm on the server:
>
> `bash curl -s ifconfig.me; echo ip -4 addr show | grep inet`
>
> If the two agree and the address does **not** start `192.168.`, `10.`, or `172.16.`–`172.31.`, it is public and this section applies. If they disagree, the machine is behind NAT — go to §10.

9.1 Before you start, have these in your hand

	Check
DNS	Both A records added **and resolving**: `dig +short crm.aspiretss.com`
SMTP	Host, user, app password
Office IP range	`curl ifconfig.me` **from the office**
Chat webhook	Google Chat space → Apps & integrations → Webhooks
Payment links	Six FastPayDirect URLs from GHL Marketing → Trigger Links

Confirm DNS resolves before you start. Let's Encrypt validates by fetching a file over HTTP from the name you claim. If DNS is not live yet, the certificate fails, Caddy serves an untrusted one, browsers refuse the site — and you will burn the Let's Encrypt rate limit rediscovering that.

9.2 The sequence

Everything below is explained afterwards. This is the order.

```
# --- on the server, as a normal user with sudo ---
sudo apt update && sudo apt upgrade -y
sudo apt install -y ufw git python3 python3-yaml curl openssl

sudo ufw allow 22/tcp && sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
sudo ufw --force enable          # SSH RULE FIRST or you lock yourself out
sudo ufw status

curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
exit                             # log out and back in, or docker stays denied
```

Log back in, then:

```
git clone https://github.com/rafilovestosuffer/aspire-crm-twenty.git
cd aspire-crm-twenty
git checkout main
git pull
git log -1 --format='%an %h %s'
```

Check the clone by author and subject, not by a remembered hash. The author must be Rafiur Rahman, and the tip must include or sit after "Deploy from main and fail if the training objects are missing". Stay on `main`.

```
./infra/init-vps-env.sh --yes
nano infra/.env                 # paste only the lines it listed

sudo ./infra/harden-host.sh     # unattended-upgrades + fail2ban. Does not touch SSH

./infra/preflight.sh            # ~20 seconds. Fix anything it calls a blocker
./infra/deploy.sh               # ~15 minutes, eleven gated steps
```

9.3 What `init-vps-env.sh` does, and the four things it cannot know

It generates the four secrets — and **will not rotate ones already set** — sets the public hostnames to `crm.aspiretss.com` / `auto.aspiretss.com`, sets `https`, empties `LIVE_MAIL_ALLOWLIST` (empty is production: consent only), clears the local alert-sink URL, and sets a real From address.

Do not copy a laptop `infra/.env` onto this machine.

> **Back up `ENCRYPTION_KEY` and `N8N_ENCRYPTION_KEY` into the password manager before going further.** No recovery path, no support to call.

You then paste four things by hand:

```
# From the office, not the server: curl ifconfig.me
N8N_EDITOR_ALLOWED_IPS=203.0.113.0/24 198.51.100.42/32

EMAIL_SMTP_USER=<workspace user>
EMAIL_SMTP_PASSWORD=<app password>
# smtp-relay.gmail.com is already set. A personal @gmail.com needs smtp.gmail.com

ALERT_WEBHOOK_URL=<real chat webhook>

# Six FastPayDirect URLs. Unset, every enrolment returns 500
ENROL_PAYMENT_LINKS=BRONZE=https://...,SILVER=https://...,GOLD=https://...,PLATINUM=https://...,DIAMOND=https://..
```

9.4 The editor allowlist — the largest avoidable risk here

`N8N_EDITOR_ALLOWED_IPS` is one line in `infra/.env`, **not** an edit to a tracked file, so it will not fight the next `git pull`. Space separated, CIDR notation; a single address is `/32`.

Public forms and inbound webhooks stay open — they have to be. **Everything else on `auto.` is the n8n editor, which holds every credential in the stack.** Exposing it to the internet is the single largest avoidable risk in this deployment.

Leave it unset and **the proxy refuses to start at all**, rather than starting with the editor open. Set it wrong and you lock yourself out rather than letting the world in. Both fail in the safer direction, and `docker compose exec caddy …` still works over SSH either way.

9.5 Rehearse the certificate before the real one

Uncomment the staging line in `infra/Caddyfile`:

```
acme_ca https://acme-staging-v02.api.letsencrypt.org/directory
```

```
./infra/up.sh --vps
docker compose -f infra/docker-compose.yml -f infra/docker-compose.vps.yml logs -f caddy
```

You want `certificate obtained successfully`. The browser **will** warn that the certificate is untrusted — that is expected from the staging CA and means the challenge worked. Staging has far looser rate limits, so a misconfigured DNS record costs you nothing here.

Then comment the staging line out and:

```
docker compose -f infra/docker-compose.yml -f infra/docker-compose.vps.yml \
  up -d --force-recreate caddy
```

Real certificates arrive within a minute. Renewal is automatic — no cron job to add, nothing to diarise.

9.6 `preflight.sh` — twenty seconds that save an evening

Checks RAM, disk, Docker, every required setting, that DNS resolves **to this server**, that ports 80 and 443 are free, and that the firewall allows them.

Every check corresponds to something that otherwise fails later with an error that does not name the cause — and a DNS mistake discovered during the real certificate request costs a Let's Encrypt rate limit.

Fix anything it calls a blocker. Warnings are judgement.

9.7 `deploy.sh` — the eleven gated steps

```
./infra/deploy.sh              # public server
./infra/deploy.sh --internal   # private network — see §10
```

#	Step
1	Start the stack — including waiting until the **server container can GET its own `SERVER_URL`**, because metadata is that HTTP call (§14.2)
2	Check the worker
3	Create the CRM workspace
4	Create the automation owner
5	Build the object model
6	Create credentials
7	Validate every CRM API call against the live schema
8	Deploy and activate — **no `--dev`**
9	Verify every layer, Caddy included
10	Prove **both** databases restore, on this machine's own disk
11	Prove the proxy rules hold

Differences from `rebuild.sh`, all deliberate: no demo data, no `--dev` workflows, the reverse proxy and real TLS included, and the backup restore verified here rather than on a laptop.

Safe to re-run. Every step is idempotent, so a failure part-way through is fixed by fixing the cause and running it again. **Do not run the Python steps by hand** — the hand list skips the restore proof and the proxy-rule proof.

Do not `docker compose up` the overlay by itself. That starts containers without the workspace, object model, credentials or workflow deploy.

Step 11 matters more than it looks: `verify_proxy_rules.py` runs the real `infra/Caddyfile` against the running stack and asserts each rule from a client outside the allowlist — the n8n editor answers **403**, while the public form, inbound webhooks and the OAuth callback go through. It then widens the allowlist by the one line an operator is told to edit and confirms the editor is served. Ten checks. **The site looks equally healthy whether that rule works or not**, which is exactly why it is proven rather than assumed. Recorded in `out/proxy_rule_verification.json`.

9.8 What `deploy.sh` does not do

It does not run `prove_workflows.py`. After the stack is up, on an **empty** CRM:

```
python3 scripts/prove_workflows.py --production --live-email you@...
```

`--production` means: no Mailpit, no failure probe, a real SMTP relay required. **Do not run it after Phase 4 data is in** — triggering the nurture would then be eligible to SMTP real leads.

Error-handler delivery is proven on the laptop with `--dev`. On the server, POST to `ALERT_WEBHOOK_URL` yourself, or wait for a real failure.

9.9 The five things to do immediately after

1. **Change the initial password.** It is in `infra/admin-initial-password.txt`, not the demo password in this repository. Change it in **both** the CRM and n8n.
2. **Make the GitHub repository private.** It is currently public.
3. **Check the padlock** on the CRM. A warning means Let's Encrypt could not reach port 80.
4. **Check you can open the automation editor.** If `N8N_EDITOR_ALLOWED_IPS` does not include the address you are browsing from, it refuses you too — by design. Fix the value and recreate Caddy.
5. **Cron the watchdog and copy the backups off the disk** — `infra/watchdog.sh` hourly, `infra/backup-offsite.sh` nightly. Same-disk dumps are not a backup.

9.10 Post-deploy verification

Check	Expected
`https://crm.aspiretss.com`	Loads, valid certificate, no warning
`https://auto.aspiretss.com/form/aspire-contact`	Form renders
`https://auto.aspiretss.com/` from an untrusted network	**403**
`curl -I http://crm.aspiretss.com`	308 redirect to https
`python3 scripts/stack_verify.py`	All green, worker **and Caddy** running
`docker compose ... ps`	`caddy server worker n8n db redis backup` all up
`python3 scripts/verify_proxy_rules.py`	Ten checks pass

Check	Expected
`python3 scripts/verify_restore.py`	Passes, on this machine's disk
`python3 scripts/prove_workflows.py --production --live-email you@...`	On an **empty** CRM only

If Twenty mailbox sync is used here, the Google OAuth redirect URIs must match `SERVER_URL` byte for byte, including `https`:

- `https://crm.aspiretss.com/auth/google/redirect`
- `https://crm.aspiretss.com/auth/google-apis/get-access-token`

Laptop `http://localhost:3000/...` URIs will not work.

`MSG Inbound Events` is public at `https://auto.aspiretss.com/webhook/mail-event`. SMTP has **no** bounce or complaint callback — a provider or a script must POST `{"type":"unsubscribe"|"bounce"|"complaint","email":"..."}`. Unsubscribe is proven in the suite; bounce and complaint need that POST wiring before they can be claimed.

---

§10 · Deploying to an internal (office-network) server

Full detail in `docs/11-internal-server-deployment.md`, which is written for someone who has never administered a server. This is the summary and the one limitation that decides whether it is the right choice.

10.1 What an internal server can and cannot do

Everything **outbound** works normally: the CRM for anyone in the office or on the VPN, sending email, scheduled automation, calling other services' APIs.

Two things **cannot** work, and no configuration fixes them:

Broken	Why
The public lead form	A visitor's browser tries to reach this server, and a private address is not routable from the internet
Inbound webhooks	Any vendor calling back cannot reach it either

This is a property of the network, not of the software. The fix is a **Cloudflare Tunnel**, which publishes only the form and webhook paths without opening a single port — §9 of `docs/11`. Do the main install first; add the tunnel when you need lead capture.

If lead capture from the website is the thing you most want to demo, read that section before you start, because it changes what you set up.

10.2 The differences from §9

	Public server	Internal server
Compose overlay	`docker-compose.vps.yml`	`docker-compose.internal.yml`
Deploy command	`./infra/deploy.sh`	`./infra/deploy.sh --internal`
Proxy config	`infra/Caddyfile`	`infra/Caddyfile.internal`
Certificates	Let's Encrypt	**Caddy's own internal CA** — must be trusted on every client machine
Hostnames	`crm.aspiretss.com`	`crm.aspire.local`, resolved by internal DNS or a hosts file
`NODE_EXTRA_CA_CERTS`	not needed	**required** — see below

10.3 The internal-CA trap

On an internal CA, Twenty's server container must **trust** its own `SERVER_URL` or every metadata request fails (§14.2). Caddy's PKI directory is `0700` and root-owned while the app runs non-root, so the root certificate must be **copied out** to `infra/certs/` — not mounted from the `caddy-data` volume. `deploy.sh --internal` does this and points `NODE_EXTRA_CA_CERTS` at the copy. `scripts/verify_proxy_rules.py` asserts it.

Every client machine must also trust that root, or browsers warn on every page.

§11 · Part D — Data migration

This is Phase 4. It has not been run. Nothing below has executed against production data; it is the procedure, and the order in it is not negotiable.

11.1 Order of operations

#	Step	Why this order
1	**Export the GHL suppression list**	Unrecoverable after termination. Independent of every other decision. Do it first, today, regardless of the timeline
2	Export the rest of GHL	Read-only puller, §4
3	Load companies	Relations resolve parent-first
4	Load people	Needs companies
5	Load opportunities	Needs both, and needs the stage map
6	Load `consentRecord`	From the suppression export. **Reconcile counts exactly**
7	Load the training objects — programs, cohorts, enrolments, webinar events and registrations	Needs people
8	Reconcile	Counts, then twenty records field by field

Batch at 60 records and respect the **100 requests/minute** Twenty limit. The scripts pace themselves; a custom loop must too.

11.2 The consent export — the highest-severity item in this project

Unsubscribe state lives in GoHighLevel. If the account closes before it is exported and loaded, previously-unsubscribed people get emailed again. That is a **compliance breach**, not an inconvenience — at a company that sells compliance.

```
python3 scripts/import_consent_csv.py unsubscribed.csv --dry-run
python3 scripts/import_consent_csv.py unsubscribed.csv
```

Gate — exact, not approximate: the number of consent records in Twenty matches the number of rows in the export. Not "about right". Exact. Record both numbers and the date.

Consent needs a named owner before any termination date is set. Today that owner is the `consentRecord` object plus rule 2 of §6.2 — every send workflow checks it. Name the person accountable for it too.

11.3 Stage mapping

None of the three GHL pipelines maps cleanly onto Twenty's defaults, and there is **no `WON` stage**. `reference/ghl_stage_map.yaml` holds the mapping. It must be reviewed by a person who sells, not only by whoever runs the migration — "Hot Lead" and "Evaluation" are commercial distinctions, not technical ones.

11.4 Field mapping

53 contact custom fields, zero opportunity custom fields. Contact-side mapping is the whole job.

Many of the 53 are one-off webinar survey questions ("5. Did the webinar meet your expectations?") that belong in a form tool, not as CRM fields. Mapping every one of them into Twenty would import GHL's mess along with its data. Decide per field: migrate, migrate-to-`formSubmission`, or drop.

The same applies to the **57 tags**. `SEG Tag Sync` classifies each one; a person confirms. Tags encoding workflow state should become fields.

11.5 Reconciliation

Gate, all three:

1. Record counts match on both sides, per object.
2. The consent count matches **exactly**.
3. Twenty records spot-checked **field by field** against the GHL UI, clean.

Compare what you hold against `totalCount` and fail if they differ. A sweep reporting "all clear" over the first page of a larger table is worse than one that errors (§15.2).

11.6 What does not migrate

- **Email template drag-and-drop structure.** `editorData` is not readable back, so only the **HTML** migrates. Rebuilding the 14 templates is manual work in whatever renders them.
- **Workflow internals.** Not available (§4.7). The six live workflows were rebuilt from evidence and need UI verification.
- **The 16 draft workflows.** They never ran. Confirm in the UI, then drop.
- **The 26 forms that have never received a submission**, and the `Form 31`… `Form 50` / `Application-prebuilt (1..3)` duplicates.
- **Conversation bodies, call recordings, submission payloads.** Never pulled, by rule. If any of these must move, that is a separate, explicitly-authorised piece of work with its own PII handling — not an extension of this one.

---

§12 · Part E — Parallel run and cutover

12.1 Parallel run

Both systems live, reconciled daily. **Two to three weeks.**

Exit criteria — all five, measured, not felt:

	Threshold
New records originating in Twenty	**≥ 90%**
Data parity between the systems	**≥ 99%**
Reporting agreement	within **2%**
Consent counts	**exact**
Automation failure rate	**< 1%**

12.2 Pre-cutover checklist

- ☐ Every surviving GHL workflow rebuilt **or explicitly marked dropped, with a reason**
- ☐ Five rebuilt workflows spot-checked by hand against the live GHL definitions (D4)
- ☐ Suppression list exported, loaded, counts reconciled exactly
- ☐ Stage map reviewed by a person who sells
- ☐ Every inbound webhook into GHL from an external system identified and repointed — **these break silently at cutover**
- ☐ Every phone number, form URL and link printed on the website, in email signatures or in contracts inventoried
- ☐ A tested restore inside the last month, date recorded
- ☐ The watchdog running and proven to alert
- ☐ A second operator named, who has completed one task from this runbook unaided
- ☐ `prove_workflows.py --production` green on the server
- ☐ `verify_proxy_rules.py` green
- ☐ Rollback plan written down and agreed

12.3 Cutover

One week. Then:

Keep GoHighLevel read-only for 30–60 days after cutover. Termination is irreversible; the cost of keeping it is trivial against discovering in week three that a form nobody inventoried was still feeding leads.

The sunset date is a **contract-termination date**, gated by the consent export and by anything with a lead time — not by when the engineering finishes. A date chosen from the engineering work alone will be wrong by roughly a month, in the dangerous direction.

12.4 Rollback

Rollback is only meaningful while GHL is still alive, which is the whole reason for the 30–60 day read-only window.

Failure	Action
Twenty unusable, GHL still live	Repoint the website form back at GHL. Leads flow again within DNS/cache time
Data corrupted in Twenty	Restore from `infra/backups/` (§13.2). Re-run the migration from GHL, which is still authoritative
An automation misfiring	Deactivate that workflow in n8n. **It is one workflow, not the stack** — this is why sends route through one place
Consent state in doubt	**Stop all sending.** Re-import from the suppression export. Do not resume until counts reconcile exactly

§13 · Part F — Operating it

13.1 The operating calendar

When	Task	Command / action
Hourly	Watchdog — stack liveness → chat webhook on failure	`infra/watchdog.sh` (cron)
Nightly	Database + volume dumps, 30-day retention	The `backup` container
Nightly	**Copy the dumps off this disk**	`infra/backup-offsite.sh user@host:/srv/aspire-crm/` (cron)
Every morning	`SYS Daily Health Check` asks whether the seven scheduled jobs ran in the last 26 hours	Automatic; it alerts
Weekly	Skim `automationRun` for FAILED, and the n8n execution list	CRM view
Monthly	**Prove a restore**	`verify_restore.py`, both databases
Monthly	Review disk on the Postgres volume	`df -h`, `docker system df`
Quarterly	Rotate the Twenty API key; review the editor allowlist	§16
Quarterly	Re-read §15 with whatever broke that quarter added to it	—
On any Caddyfile edit	Re-prove the proxy rules	`verify_proxy_rules.py`
On any workflow change	Regenerate → validate → deploy → prove	§7.4, §8.7

13.2 Backups and restore

Nightly dumps of **both** databases and the Twenty / n8n file volumes into `infra/backups`, 30-day retention. Attachments are not in Postgres — **a database-only restore leaves records pointing at files that are gone.**

That directory is on the same disk as the database, which is **not a backup**:

```
./infra/backup-offsite.sh user@backup-host:/srv/aspire-crm/
# or: rsync -az --delete infra/backups/ backup-host:/srv/aspire-crm/
```

Proving a restore — monthly, and the date recorded:

```
python3 scripts/verify_restore.py
python3 scripts/verify_restore.py --database n8n
python3 scripts/verify_restore.py --keep       # leave the scratch copy for inspection
```

It dumps a database, restores it into a **scratch** database, counts rows in both, compares exactly, then drops the scratch copy. Last verified run: Twenty **4,045 rows** and n8n **1,364 rows**, restored exactly. The date of the last success is recorded in `out/restore_verification.json`.

An untested backup is not a backup. A backup job that has silently died is the worst failure in this system, because everything looks fine until the day it does not.

13.3 The watchdog

Cron `infra/watchdog.sh` hourly. It runs `stack_verify.py` and POSTs to `ALERT_WEBHOOK_URL` if any layer is down.

A dead worker is completely silent without this. The UI stays healthy; scheduled work simply stops.

```
crontab -e
# 7 * * * * cd /home/rafi/aspire-crm-twenty && ./infra/watchdog.sh >> /tmp/watchdog.log 2>&1
# 30 2 * * * cd /home/rafi/aspire-crm-twenty && ./infra/backup-offsite.sh user@backup-host:/srv/
aspire-crm/
```

Prove the alert path works — POST to the webhook by hand — rather than assuming silence means health.

13.4 Monitoring — what to watch and why

Signal	Why it matters
Twenty worker alive	The UI stays healthy while scheduled work and mailbox sync silently stop
Workflow failure rate	A rising rate means something upstream changed
Executions per hour per workflow	Catches a runaway loop before it exhausts the API
Disk on the Postgres volume	Execution data grows fast; pruning is set to 14 days
Backup freshness	See above
Certificate expiry	Automatic, but worth an eye after any DNS change

13.5 Routine operator tasks

A paid enrolment. The workflow will not mark anything paid. After FastPayDirect clears:

```
python3 scripts/mark_enrolment_paid.py --email student@example.com --yes
```

A new suppression import:

```
python3 scripts/import_consent_csv.py unsubscribed.csv --dry-run
python3 scripts/import_consent_csv.py unsubscribed.csv
```

Changing the editor allowlist: edit `N8N_EDITOR_ALLOWED_IPS` in `infra/.env`, recreate Caddy, then re-run `verify_proxy_rules.py`.

Upgrading Twenty or n8n. The tags (`TAG`, `N8N_TAG`) are pinned in `infra/.env` and stay pinned. To upgrade: read the release notes → verify a restore → bump the tag → `up -d` → `stack_verify.py` → `prove_workflows.py`. In that order, and never on a Friday.

Changing an automation. Never in the n8n UI. Edit `scripts/build_n8n_workflows.py`, then:

```
python3 scripts/build_n8n_workflows.py
python3 scripts/validate_workflow_queries.py
python3 scripts/n8n_deploy.py --activate
python3 scripts/prove_workflows.py --only <suite>
git add n8n/workflows && git commit
```

13.6 Diagnostics — the first five commands

When something is wrong and you do not yet know what:

```
# 1. Is everything running?
docker compose -f infra/docker-compose.yml -f infra/docker-compose.vps.yml ps

# 2. Does every layer answer independently?
python3 scripts/stack_verify.py --verbose

# 3. What has been failing?
docker compose -f infra/docker-compose.yml -f infra/docker-compose.vps.yml \
  logs -f --tail=100 server worker n8n caddy

# 4. Can the server reach its own public URL?  (§14.2 — the misleading one)
docker compose -f infra/docker-compose.yml -f infra/docker-compose.vps.yml \
  exec server curl -sS -o /dev/null -w '%{http_code}\n' "$SERVER_URL/healthz"

# 5. Is there disk and memory?
df -h; free -h; docker system df
```

Then go to §14 with a symptom rather than a feeling.

---

§14 · Troubleshooting

14.1 Build and stack

Symptom	Cause and fix
Twenty never becomes healthy	First boot runs migrations; give it 5 minutes. Then `docker compose … logs server`
Login redirect loop	`SERVER_URL` does not match how you reach it, **scheme included**
`password authentication failed`	The Postgres password is baked into the volume. Restore the original value, or `./infra/up.sh --destroy` and start over
Scheduled workflows never fire	Worker container down. `docker compose … logs worker` — **the UI stays healthy regardless**
`permission denied` on docker	You did not log out and back in after `usermod -aG docker`
Sluggish past ~10k records	`shm_size` and Postgres tuning are already set; confirm they applied. Then check RAM — this wants 8 GB
Docker installed but daemon unreachable	Docker Desktop not started, or WSL integration off on Windows
Twenty API returns 429	100 requests/minute. The scripts pace themselves; a custom loop must too
n8n stuck in "starting" forever	The healthcheck must hit `127.0.0.1:5678/healthz`, **not** `localhost`. The container listens on IPv4 only; `localhost` often resolves to `::1` and the check fails forever while the API is fine
n8n cannot reach Twenty	Inside Docker the address is `http://server:3000`, not `localhost:3000`

14.2 The metadata 500 — read this before blaming the API key

`/rest/metadata/*` **is not served in-process.** Twenty makes an HTTP call to `${SERVER_URL}${path}` from **inside** the server container and returns the result. `getServerUrl` prefers the environment variable over the request, so `SERVER_URL` always wins when it is set.

If the server cannot **resolve, reach or trust** its own `SERVER_URL`, every metadata request answers **HTTP 500 with an empty body and no server log line** — and the provisioner, the query validator and the stack check all fail at once, none of them naming the cause. **The ordinary REST API keeps working throughout**, which is what sends people looking at the API key.

Ask the server directly:

```
docker compose -f infra/docker-compose.yml -f infra/docker-compose.vps.yml \
  exec server curl -sS -o /dev/null -w '%{http_code}\n' "$SERVER_URL/healthz"
```

What you see	What it means
`could not resolve host`	The compose network has no alias for the public name. The `caddy` service in the overlay sets one — check it is not commented out. Without it, the server reaches its own public URL only if the provider supports hairpin NAT

What you see	What it means
`certificate verify failed`	On a public server this usually means **ACME has not finished** — until it does, Caddy answers with a self-signed certificate the server does not trust. `docker compose logs caddy` says whether the challenge succeeded. Fix DNS or port 80, wait for the certificate, re-run `deploy.sh`. Internally, the certificate is always Caddy's own — which is why `deploy.sh --internal` copies its root into `infra/certs/` and sets `NODE_EXTRA_CA_CERTS`
`connection refused`	Caddy is down, or `SERVER_URL` names a host that is not it
`200`	The self-call is fine; the problem is elsewhere

Asserted by `scripts/verify_proxy_rules.py` and gated in `infra/deploy.sh`.

14.3 Workflows and queries

Symptom	Cause and fix
A query returns far too many rows	Almost certainly the `filter[field][op]=` shape — Twenty **ignores it and returns the whole table**. Run `validate_workflow_queries.py`
A scan misses records that exist	The 200-row page ceiling. It returns 200 with HTTP 200 and no warning. Page with `starting_after` (§15.2)
An order-by has no effect	`orderBy=` is silently ignored. It is `order_by=field[DescNullsLast]`
A filter with a space or bracket 400s	Every interpolated value needs `encodeURIComponent` (§15.4)
A date filter or write body 400s	Twenty accepts `…Z` or a bare date. An offset like `…-05:00` is a **400 in filters and in write bodies**
n8n form 404 on the production URL	The workflow must be **saved and active**; check `WEBHOOK_URL` matches `N8N_PUBLIC_URL`
Form submits but every field is null	Fields post as `field-0`, `field-1`… **by position, not by label**. Only affects scripted posts; a browser gets it right
Form URL is a UUID instead of `/form/aspire-contact`	The path moved into `options` at typeVersion 2.2. The generator sets both — regenerate rather than editing in the UI
"references workflow X which is not published"	Sub-workflow and error-workflow links resolve by **id, not name**. Deploy through `n8n_deploy.py`, never by importing JSON
An expression fails with a syntax error	A nested `}}` closed it early. Emit `} }`. The generator lints for this
`jsonBody` not evaluated	The **whole** parameter must start with `=`
A Code node cannot see an env var	Code nodes run in a sandboxed task runner: use `$env`, never `process.env`
Error workflow never fires	`settings.errorWorkflow` fires **only for production executions**. Testing failure handling from the editor proves nothing
A webhook returned 200 but nothing happened	A webhook answers on receipt, before the workflow runs. **Assert on the record written, not the HTTP status**
Task body rejected with 400	Task body is `bodyV2`, a RICH_TEXT composite taking `{"markdown": "..."}`. A bare string is a 400; a plain `body` key is not a field

14.4 Mail

Symptom	Cause and fix
Gmail refuses the login	Google may flag a first-time `smtp.gmail.com` login from a new IP. Check the account's security challenge, not the workflow
Gmail rejects the From	It must be the authenticated account or an allowed send-as alias
`smtp-relay.gmail.com` rejects a personal address	It is Workspace-only. Use `smtp.gmail.com`
A send is refused and logged `Blocked: not_on_allowlist`	Working as designed. `LIVE_MAIL_ALLOWLIST` is the second gate next to consent. Empty = production
A send is refused and logged as a consent block	Working as designed
Bounces never write back	SMTP has no bounce webhook. Either wire a provider to POST `/webhook/mail-event`, or parse the bounce mailbox with an IMAP trigger

14.5 GoHighLevel

Symptom	Cause
An error that reads like auth	The missing `Version: 2021-07-28` header
Cloudflare **1010**	Default Python-urllib User-Agent. Send one
A count of **0** with HTTP 200	Check `list_key` against a real response before believing it. This is the worst failure mode in the repo
`/calendars/events` 422	Refuses a location-wide query. Fan out per calendar
`/surveys/` 422	`limit` caps at 50
`/medias/files` 422	Requires an undocumented `type` parameter
403 on `/locations/search`	A sub-account token cannot discover its own location id. Take it from the browser URL
The capture script stops mid-run	The session JWT expired. Re-capture the cURL and run again. **Normal, not a failure**

14.6 Certificates and proxy

Symptom	Cause
Certificate fails	Almost always DNS not resolving, or port 80 blocked. Caddy's logs say which. It retries on its own — do not restart it
Certificate warning on an internal server	The client machine does not trust Caddy's root, or the browser was not restarted
The site loops on login	`SERVER_URL` versus the address in the browser — usually `http` vs `https`, or a missing subdomain
The editor is reachable from anywhere	`N8N_EDITOR_ALLOWED_IPS` wrong or the Caddyfile edited. **Treat as an incident** — that host holds every credential in the stack. Fix, recreate Caddy, re-run `verify_proxy_rules.py`, and rotate the credentials

Symptom	Cause
Caddy will not start	The allowlist is unset. Deliberate — it refuses rather than starting with the editor open

§15 · The quirks that cost us time

Every entry here is a defect that was **silent** — HTTP 200 and a wrong answer, or a green suite over broken code. They are reproduced here so an operator never has to rediscover one. `CLAUDE.md` is the authoritative copy.

15.1 Twenty REST query syntax

Purpose	Correct	Wrong, and what happens
filter	`?filter=field[eq]:value`	`?filter[field][eq]=value` — **ignored, returns the whole table**
multiple	`?filter=a[eq]:1,b[eq]:2` (comma = AND)	—
in	`?filter=status[in]:[A,B]`	bare comma list → 400
substring	`?filter=domainName.primaryLinkUrl[ilike]:%acme%`	—
order	`?order_by=field[DescNullsLast]`	`orderBy=` — **silently ignored**
relation	`?filter=personId[eq]:<uuid>`	the relation is `person`, the filter key is `personId`
page size	`?limit=200` is the **hard ceiling**	`?limit=500` — **returns 200, HTTP 200, no warning**
datetime	`2026-08-16T21:12:12.072Z` or `2026-08-16`	`…-05:00` / `…+00:00` — **400, in filters and write bodies**

Enforced by `scripts/validate_workflow_queries.py`. Run it after touching any workflow.

15.2 A page is 200 rows and nothing raises that

Any scan that can outgrow 200 rows **must page**: `?starting_after=<pageInfo. endCursor>` until `pageInfo.hasNextPage` is false. Verified live — pages do not overlap and sum to `totalCount` exactly.

Compare what you hold against `totalCount` and fail if they differ. A sweep reporting "all clear" over the first page of a larger table is worse than one that errors.

In n8n this is the HTTP Request node's pagination option, which emits **one item per page**. `scripts/prove_paging.py` pushes a table past 200 and asserts the generated node read every row.

15.3 Dates go to Twenty as UTC with a `Z`, always

Luxon's `toISO()` renders in the workflow's zone, so `$now.toISO()` is only a `Z` while `TZ=UTC`. Set `TZ=America/New_York` in `infra/.env` — an entirely reasonable thing for a US company to do — and **every `automationRun` write, every `sentAt`, every task `dueAt` and the renewal scan start failing at once, on a stack that passed every test.**

`.toUTC().toISO()` is byte-identical today and correct in any zone. The generator refuses to build without it.

15.4 Every interpolated filter value must be `encodeURIComponent`-wrapped

A space, bracket or colon corrupts the query into a 400. This broke the error handler's **own** dedupe query on a workflow name containing brackets: the failure was logged, the handler then errored, and **the alert was never sent**.

Values reaching these filters include user-typed emails and a slug taken off a public URL. The generator lints for it.

15.5 Metadata is a slice, not a schema

`/rest/metadata/*` **rejects every query string**, and `/rest/metadata/objects` embeds only a slice of each object's fields. It is not a complete schema. Merge it with the keys of a real row from `/rest/{object}?limit=1`.

And see §14.2 for why it can return 500 with nothing in the logs.

15.6 n8n

- An expression ends at the **first** `}}`. A nested object literal (`{"a": {"b": $x}}`) closes it early and the remainder is a syntax error. Emit `} }`.
- `jsonBody` is only evaluated when the **whole** parameter starts with `=`.
- Sub-workflow references and `settings.errorWorkflow` resolve by **id, not name**. Named, they dangle silently — every workflow looks like it has error handling and none of it fires.
- Form Trigger inputs post as `field-0`, `field-1`… **by position**, not by label. The form path lives in `path` at typeVersion ≤ 2.1 and in `options.path` from 2.2 — set both, or the URL falls back to a random UUID.
- Code nodes run in a sandboxed task runner: use `$env`, never `process.env`.
- A Switch's `fallbackOutput` must be an existing index or `"extra"`.
- **`settings.errorWorkflow` fires only for PRODUCTION executions.**
- A webhook answers on receipt, before the workflow runs. A 200 says nothing about the outcome.

15.7 Twenty data-model facts

- Task body is `bodyV2`, a RICH_TEXT composite taking `{"markdown": "..."}`.
- Default opportunity stages are `NEW, SCREENING, MEETING, PROPOSAL, CUSTOMER`. **There is no `WON`.**
- `seed_demo_data.py --wipe` matches on seeded parents — company, person **and training account**. Matching on `companyId` alone misses consent records and phishing baselines.

15.8 GoHighLevel

Reproduced in §4.5–§4.6 and §14.5. The one to memorise: **a silent zero behind an HTTP 200 is the worst failure mode in this repository.**

---

§16 · Security, privacy and compliance

16.1 The governance position

The audit needs **structure, not people**. No contact records, no message bodies, no call recordings, no submission payloads. This is both a governance position and a practical one: Aspire sells compliance, and the audit should meet the standard Aspire sells.

Enforced structurally, not by discipline:

- The puller issues **GET only** and refuses other methods at the transport layer.
- Endpoints that could carry PII are classed `volume` in `reference/ghl_endpoints.yaml`: **bodies are discarded at pull time**; only counts, field names and categorical histograms are written to disk.
- Tokens are redacted from all error output.
- `.env`, `infra/.env`, `raw/`, `workflows/`, `.session_curl.txt` are gitignored.
- Rate limited to 80 requests / 10s against a 100/10s ceiling.

16.2 Secrets

Rule	
`ENCRYPTION_KEY`, `N8N_ENCRYPTION_KEY`	**Backed up off-host**, in the password manager. No recovery path
Twenty API key	Scoped to a role, **not admin**. Rotated quarterly
GHL token	Deleted when the audit finished. Rotation guidance is 90 days with a 7-day overlap
Credentials in n8n	**Always** the credential store. Never inline in a node
Default logins	`admin@aspiretss.com` / `AspireDemo2026!` is **public in this repository**. Changed on the server as deploy step §9.9.1
This runbook	Contains no secret and never may

16.3 Network exposure

Surface	Exposure
CRM (`crm.`)	Public, TLS, authenticated
Public form, `/webhook/*`, OAuth callback on `auto.`	**Public by necessity**
n8n editor on `auto.`	**IP allowlist only.** Holds every credential in the stack
Postgres, Redis	Never reachable from outside the host
SSH	Key auth only, password auth off
Everything else	Denied by `ufw`

Open item: the GitHub repository is still public. It contains no secrets — that is enforced — but it does contain the demo credentials, the hostnames and the whole architecture. Make it private (§9.9.2).

16.4 Two environments, not one

Two n8n instances, or at minimum two credential sets. **A test execution that emails a real client is a career-grade mistake and it happens on day one.** The allowlist gate (`LIVE_MAIL_ALLOWLIST`) is the structural version of this control: on a laptop, seeded `.example.com` addresses cannot leave the host even if a workflow is triggered by hand.

16.5 Consent as a control, not a field

Restating §6.2 because it is the compliance-relevant one:

1. Populated from the GHL export **before** termination.
2. **Every** send checks it — enforced in one place, `VEND Send Email`.
3. Inbound unsubscribes and bounces write back, or it drifts within weeks.

A consent record nothing consults provides no protection and false assurance, which is worse than none.

---

§17 · Handover and the second operator

17.1 The risk, stated plainly

Right now this project has one point of failure and it is a person. That is on the risk register for a reason, and it is the most likely thing to actually hurt us — more likely than any technical failure in this document.

17.2 The exit criterion

A second operator is not "someone who has been shown the system". It is:

A named person who has completed at least one task from this runbook, unaided, on a real system, and whose work was verified.

Suggested first tasks, in increasing order of consequence:

1. Build the whole stack on their own laptop from §8.1 and get a green `prove_workflows.py`.
2. Run `verify_restore.py` on the server and record the date.
3. Change the editor allowlist and re-prove the proxy rules.
4. Regenerate, validate, deploy and prove a trivial workflow change.
5. Walk the demo in §8.11 in front of someone.

17.3 What the second operator must be able to answer

- Where are `ENCRYPTION_KEY` and `N8N_ENCRYPTION_KEY` backed up?
- How do you tell whether the worker is alive, and why does the UI not tell you?
- What is the last date a restore was proven, and where is that recorded?
- Why does a Twenty query that returns the whole table not raise an error?
- Which single workflow is allowed to send email, and what does it check first?
- What happens to consent state if GoHighLevel is terminated tomorrow?

If they can answer those six, they can hold the system.

17.4 Hard questions, and the honest answers

"Is this just a hobby project that will break?" Every claim in this document is produced by a script that runs against the live system. There are three gates in the build: the generator refuses to emit code that would fail at runtime, a validator checks every CRM API call against the live schema, and 25 checks exercise the real paths. They caught more than twenty defects, several of which would have been silent in production.

"What happens when Rafi is on holiday?" Today, that is a real risk. §17.2 is the fix and it is a Phase 5 gate. Do not skip it.

"What if Twenty disappears?" The data is in Aspire's own Postgres. It can be dumped to CSV or SQL at any time. That is strictly better than the current position, where the data is in someone else's SaaS.

"Why not just keep GoHighLevel?" Because it cannot model the training funnel as objects; because it will not tell us what its own automation did; and because it costs a recurring fee for a system Aspire cannot inspect.

§18 · Status — measured, not asserted

Regenerate rather than trusting these numbers after any change:

```
python3 scripts/implementation_audit.py
python3 scripts/implementation_audit.py --markdown   # → out/implementation_status.md
```

18.1 What is built

Objects in the CRM	**59** — 28 Twenty standard, **31 custom**
Fields and relations provisioned	**198**, from a version-controlled schema
Demo dataset	**448** records, deterministic
Automations	**17** live, plus 2 that exist only to test the others

18.2 Feature disposition — all 111 catalogued

Status	Count	Meaning
LIVE	24	Verified working on the running stack
NATIVE	13	Twenty or n8n does it out of the box; needs configuration
MODELLED	22	The object exists to hold the data; no automation on it yet
DESIGNED	19	Specified in the migration guide; nothing built
DEFERRED	33	Deliberately out of scope, each with a recorded reason

Working or native today: 37 of 111 (33%). Nothing is "built but never run".

That last sentence is the one that matters. The dangerous category in any migration is code that exists, looks finished, and has never executed. **It is empty here.**

18.3 What the 33 deferred actually are

Mostly telephony (SMS, voice, IVR, call recording — licensed carrier required, and the account shows light use), website and funnel hosting (aspiretss.com stays where it is), and course delivery (aspireelearning.com already does it).

They are not gaps. They are decisions, each with a reason in `out/implementation_status.md`.

18.4 What has been rehearsed rather than assumed

- **The deploy.** `infra/deploy.sh` ran end to end — all eleven steps, in order, on a stack brought up from the same compose files the server uses. It found two things that would each have stopped the real deploy dead, and both are fixed: the reverse proxy config, and the metadata self-call (§14.2).
- **The restore.** Both databases, exact row counts, recorded in `out/restore_verification.json`.
- **The proxy rules.** Ten checks from a client outside the allowlist, recorded in `out/proxy_rule_verification.json`.
- **The workflow library** — 25 checks against a running stack.

18.5 What has not been verified against a live stack

Honesty about the current edge:

- The six objects and eight workflows added after the 18 August realignment were generated and linted (19 workflows clean, 85 Twenty calls validated `--offline`, 48 write bodies checked against the schema, negative controls caught) — but `--offline` checks the schema **file**, not a running instance. It cannot see drift or anything provisioned by hand, and it says so in its own output rather than claiming a live pass.
- **Still required before those are trusted:** `twenty_provision.py` against a real instance, then `validate_workflow_queries.py` **without** `--offline`, then `prove_workflows.py`. The `webinar`, `enrol`, `ai` and `tags` proof suites have never been executed.
- Bounce and complaint handling. Only the scripted unsubscribe path is proven.
- Performance at real data volume.

---

§19 · Cost, roadmap and the things that can hurt

19.1 Cost

Twenty, n8n, Postgres, Redis	**£0** — open source, self-hosted
Server	One VPS. 4–8 cores, 8–16 GB for real load
Email under ~2,000 recipients/day	**£0** — the existing Google Workspace relay
Email above that	An SMTP relay. Amazon SES is about **$0.10 per 1,000 emails**
Booking pages	Cal.com, self-hosted, £0
Quotes and e-signature	DocuSeal, self-hosted, £0 — eIDAS/ESIGN valid

There is no irreducible third-party product for what is in scope. Every GoHighLevel capability Aspire uses has a native answer, an n8n answer, or a self-hostable one. The recurring GHL subscription is replaced by a server bill.

The one number to measure before go-live is send volume, because it decides whether the Workspace relay is enough. Google Workspace allows roughly 2,000 external recipients per day; Microsoft 365 around 10,000. A few thousand B2B contacts fits. A consumer-scale list does not.

Two caveats on bulk email through the mailbox:

- **What is genuinely missing is bounce and complaint feedback.** SMTP gives no webhook. Mitigation: parse the bounce mailbox with n8n's IMAP trigger on a schedule. Not as good as an ESP webhook, but not nothing.
- **Sending campaign volume from the mailbox that also carries real sales conversations puts both at risk** if reputation slips. Use a subdomain sender beyond a few hundred a day.

19.2 Roadmap

Each phase has a gate. Do not start the next until it passes.

Phase	Work	Time	State
1	Prove the automation layer	—	**done**
2	Native Twenty config — connect a mailbox, saved views, roles	3 days	next
3	Cal.com (booking) + DocuSeal (quotes, e-signature)	1 week	
4	Data migration — **export the suppression list first**	1 week	
5	Production hardening — queue mode, monitoring, runbook, **second operator**	3 days	TLS and a tested restore already done
6	Parallel run, reconcile daily, then cut over	2–3 weeks	

6–8 weeks to production, with a demonstrable system from week one.

19.3 The three things that can actually hurt

1. The consent list is unrecoverable. GoHighLevel's suppression list does not come back after the account is terminated. Export it and reconcile the count exactly, before any termination date is agreed.

2. One operator. §17.

3. Silence that reads as health. A dead worker, a backup job that died, a filter Twenty ignored, a count of zero behind an HTTP 200. Every gate and every proof in this system exists because of this class of failure. **When something looks fine, check that the thing which would tell you otherwise is itself alive.**

19.4 Sequencing risks, if the deferred items ever come back

These do not scale with effort, and each has burned real migrations:

Item	Lead time	Why it cannot be rushed
A2P 10DLC registration	Weeks	Carrier-side regulatory review. Unregistered traffic is filtered or blocked
Phone number porting	Days–weeks	Numbers can be unreachable during the window. Irreversible on a short timescale
Email domain warm-up	2–6 weeks	**You cannot switch sending domains on Monday and campaign on Tuesday**
Consent export	Must precede termination	See above
DNS cutover	Hours–days	Choreographed with whatever replaces the hosting

§20 · Appendices

A · Command index

Audit

Command	Does
`python scripts/ghl_pull.py --dry-run`	Show the plan, call nothing
`python scripts/ghl_pull.py --only <endpoint>`	One endpoint
`python scripts/ghl_pull.py [--window-days N] [--force]`	Full discovery run
`python scripts/ghl_workflow_capture.py [--dry-run] [--limit N] [--published-only]`	Workflow internals via the session route
`python scripts/build_audit.py [--min-records N]`	→ `out/feature_audit.csv`
`python scripts/twenty_probe.py [--expect a,b,c]`	→ `out/twenty_capability.md`

Build

Command	Does
`./infra/up.sh [--no-dev|--vps|--down|--destroy]`	Stack lifecycle
`./infra/rebuild.sh [--destroy]`	Zero to proven, eleven gates
`python scripts/stack_verify.py [--verbose]`	Every layer independently
`python scripts/bootstrap_workspace.py [--show]`	Twenty user, workspace, API key
`python scripts/bootstrap_n8n.py`	n8n owner + API key
`python scripts/twenty_provision.py [--dry-run|--only X|--objects-only|--skip-relations]`	The object model
`python scripts/seed_demo_data.py [--dry-run|--wipe]`	448 demo records
`python scripts/n8n_credentials.py [--dry-run]`	Twenty header auth + SMTP
`python scripts/build_n8n_workflows.py [--list]`	Generate the workflow library
`python scripts/validate_workflow_queries.py [--offline|--quiet]`	Every Twenty call vs the schema
`python scripts/n8n_deploy.py [--dry-run|--dev|--activate|--only X]`	Deploy workflows

Prove

Command	Does
`python scripts/prove_workflows.py [--only X|--keep|--live-email A|--production]`	25 checks against the running stack

Command	Does
`python scripts/prove_live_mail.py [--nurture|--cleanup|--live-email A]`	Operator-shaped live mail run
`python scripts/prove_paging.py`	Pushes a table past 200 rows
`python scripts/verify_proxy_rules.py`	Ten proxy rules from outside the allowlist
`python scripts/verify_restore.py [--database n8n|--keep]`	Dump, restore, compare row counts
`python scripts/implementation_audit.py [--markdown]`	Per-feature status

Deploy and operate

Command	Does
`./infra/init-vps-env.sh --yes`	Secrets + public hostnames
`sudo ./infra/harden-host.sh`	unattended-upgrades + fail2ban
`./infra/preflight.sh`	Is this server ready?
`./infra/deploy.sh [--internal]`	Eleven gated deploy steps
`./infra/watchdog.sh`	stack_verify → chat webhook on failure
`./infra/backup-offsite.sh user@host:/path`	Copy dumps off this disk
`python scripts/import_consent_csv.py FILE [--dry-run]`	Suppression list → `consentRecord`
`python scripts/mark_enrolment_paid.py --email X --yes`	`PAYMENT_SENT` → `PAID`

B · Gate index — every point where you stop

#	Gate	Where
1	GHL auth returns the location object	§4.4
2	Coverage report read before trusting `raw/`	§4.5
3	Five audit rows spot-checked by hand	§4.9
4	`stack_verify.py` all green	§8.3
5	Object model probe reports the training objects	§6.4
6	No view shows an empty state	§8.5
7	Generator emits without lint failures	§8.7
8	Query validator green **against a live instance**	§8.7
9	`prove_workflows.py` — 25 checks	§8.7
10	`preflight.sh` blockers cleared	§9.6
11	Restore proven on the machine that holds the data	§9.7, §13.2
12	Proxy rules proven from outside the allowlist	§9.7

#	Gate	Where
13	Consent counts reconcile **exactly**	§11.2
14	Twenty records spot-checked field by field	§11.5
15	Five rebuilt workflows checked against the GHL UI	§12.2
16	Parallel-run exit criteria, all five	§12.1
17	A second operator has completed one task unaided	§17.2

C · Glossary

Term	Means
GHL	GoHighLevel — the system being replaced
Twenty	The open-source CRM, self-hosted. The system of record
n8n	The open-source automation engine. Orchestration only
Caddy	The reverse proxy — TLS, routing, and the editor IP allowlist
Mailpit	Local SMTP catcher. Laptop only, behind a compose profile
PIT	Private Integration Token — GHL's static bearer token
Tier 0–6	The six audit stages (§4.2)
Disposition	`TWENTY` · `N8N` · `TWENTY+N8N` · `DROP` · `ESCALATE` · `UNKNOWN`
ESCALATE	Cannot be done by Twenty or n8n alone; needs a vendor and a budget
`automationRun`	The CRM object holding every workflow execution, success and failure
`consentRecord`	Per-person, per-channel consent state with source and proof
A2P 10DLC	US carrier registration required for application-to-person SMS
Gate	A point where you stop until it passes. There are no optional gates
FastPayDirect	The off-platform payment link provider for bootcamp enrolment

D · Change control for this document

- This file is version-controlled. Change it in a commit, with a reason.
- **When a procedure changes, this runbook changes in the same commit.** An out-of-date runbook is worse than none, because it is trusted.
- When something breaks in a way §14 did not predict, add the symptom to §14 and the cause to §15 **before** closing the incident. That is how §15 got every entry it has.
- Numbers in §18 are generated. Re-derive them with `scripts/implementation_audit.py`; do not hand-edit them.
- Nothing secret enters this file, ever.

End of runbook. If something in here is wrong, fix it in the repository rather than remembering the correction — the next person to read it will not have been in the room.